

SOLIDWORKS API (.NET VBA C#) — Examples (29/50):

Get_Language_and_Localized_Menu_Names_Example_VBNE

Get_Mates_and_Mate_Entities_Example_VBNE

SOLIDWORKS API Documentation (converted from CHM)

Get Language and Localized Menu Names Example_VB (Topic)

SOLIDWORKS API Help

Get Language and Localized Menu Names Example (VBA)

This example shows how to get the current language used by the SOLIDWORKS Design application and the names of some of the menus in the local language.

```
'-----
' Preconditions: Open the Immediate window.
'
' Postconditions: Examine the Immediate window.
'-----
Option Explicit

Sub main()
    Dim swApp As SldWorks.SldWorks

    Set swApp = Application.SldWorks
    Debug.Print "Language = " & swApp.GetCurrentLanguage
    Debug.Print " File Menu      = " + swApp.GetLocalizedMenuName(swFileMenu)
    Debug.Print " Edit Menu      = " + swApp.GetLocalizedMenuName(swEditMenu)
    Debug.Print " View Menu       = " + swApp.GetLocalizedMenuName(swViewMenu)
    Debug.Print " Insert Menu     = " + swApp.GetLocalizedMenuName(swInsertMenu)
    Debug.Print " Tools Menu      = " + swApp.GetLocalizedMenuName(swToolsMenu)
    Debug.Print " Window Menu     = " + swApp.GetLocalizedMenuName(swWindowMenu)
    Debug.Print " Help Menu       = " + swApp.GetLocalizedMenuName(swHelpMenu)
    Debug.Print " View Toolbars Menu = " + swApp.GetLocalizedMenuName(swViewToolbarsMenu)
End Sub

'Language = english
'File Menu = &File
'Edit Menu = &Edit
'View Menu = &View
'Insert Menu = &Insert
'Tools Menu = &Tools
'Window Menu = &Window
'Help Menu = &Help
'View Toolbars Menu = &Toolbars

'Language = spanish
'File Menu = &Archivo
'Edit Menu = &Edici3n
'View Menu = &Ver
'Insert Menu = &Insertar
'Tools Menu = &Herramientas
'Window Menu = Ven&tana
'Help Menu = &?
'View Toolbars Menu = Barras de &herramientas

'Language = french
'File Menu = &Fichier
'Edit Menu = &Edition
'View Menu = Affic&hage
'Insert Menu = &Insertion
'Tools Menu = &Outils
'WindowMenu = Fe&n4tre
'Help Menu = &?
'View Toolbars Menu = &Barre d'outils

'Language = german
'File Menu = &Datei
'Edit Menu = &Bearbeiten
'View Menu = &Ansicht
'Insert Menu = &Einfügen
'Tools Menu = E&xtras
'Window Menu = &Fenster
'Help Menu = &Hilfe
'View Toolbars Menu = S&ymbolleisten

'Language = italian
'File Menu = &File
'Edit Menu = &Modifica
'View Menu = &Visualizza
'Insert Menu = &Inserisci
'Tools Menu = &Strumenti
'Window Menu = Fi&n4stra
'HelpMenu = &?
'View Toolbars Menu = &Barre degli strumenti

'Language = japanese
'File Menu = I&S2U(&F)
'Edit Menu = •O&W(&E)
'View Menu = •Z!(&V)
'Insert Menu = '}'ü(&I)
'Tools Menu = Å°Ü(&T)
'Window Menu = 3~Y&P3(&W)
'HelpMenu = İÜİ&(&H)
'View Toolbars Menu = Å°Ü&P°(&T)
```

Get Last Save Error Example (C#)

This example shows how to get the last save error issued by Microsoft in the current SOLIDWORKS Design session.

```
//-----  
// Preconditions:  
// 1. Open a SOLIDWORKS Design session and cause Microsoft to issue a save error.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets the last save error issued by Microsoft.  
// 2. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
  
namespace Macro1CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        object varPath;  
        object varError;  
        object varMessage;  
        long err;  
  
        public void Main()  
        {  
            varMessage = swApp.GetLastSaveError(out varPath, out varError);  
            if (!(varError == null))  
            {  
                err = (long)varError;  
                if (!(err == 0))  
                {  
                    Debug.Print("Document generating the save error: " + (String)varPath);  
                    Debug.Print("Error code: " + (String)varError);  
                    Debug.Print("Error message: " + (String)varMessage);  
                }  
            }  
            else  
            {  
                Debug.Print("This SOLIDWORKS Design session has not experienced a save error.");  
            }  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Get Last Save Error Example (VBA)

This example shows how to get the last save error issued by Microsoft in the current SOLIDWORKS Design session.

```
'-----  
' Preconditions:  
' 1. Open a SOLIDWORKS Design session and cause Microsoft to issue a save error.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the last save error issued by Microsoft.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim varPath As String  
Dim varError As Long  
Dim varMessage As String  
Sub main()  
  
    Set swApp = Application.SldWorks  
    varMessage = swApp.GetLastSaveError(varPath, varError)  
    If Not varError = 0 Then  
        Debug.Print "Document generating the save error: " & varPath  
        Debug.Print "Error code: " & varError  
        Debug.Print "Error message: " & varMessage  
    Else  
        Debug.Print "This SOLIDWORKS Design session has not experienced a save error."  
    End If  
End Sub
```

Get Last Save Error Example (VB.NET)

This example shows how to get the last save error issued by Microsoft in the current SOLIDWORKS Design session.

```
' .....  
' Preconditions:  
' 1. Open a SOLIDWORKS Design session and cause Microsoft to issue a save error.  
' 2. Open the Immediate window.  
' .....  
' Postconditions:  
' 1. Gets the last save error issued by Microsoft.  
' 2. Examine the Immediate window.  
' .....
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Dim varPath As Object  
    Dim varError As Object  
    Dim varMessage As Object
```

Sub main()

```
    varMessage = swApp.GetLastSaveError(varPath, varError)  
    If Not varError = 0 Then  
        Debug.Print("Document generating the save error: " & varPath)  
        Debug.Print("Error code: " & varError)  
        Debug.Print("Error message: " & varMessage)  
    Else  
        Debug.Print("This SOLIDWORKS Design session has not experienced a save error.")  
    End If
```

End Sub

Public swApp As SldWorks

End Class

Get Layers Example (C#)

This example shows how to get the layers in a drawing document.

```
//-----
// Preconditions:
// 1. Open a drawing document.
// 2. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetLayers_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            LayerMgr swLayerMgr = default(LayerMgr);
            object[] vLayerArr = null;
            object vLayer = null;
            Layer swLayer = default(Layer);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swLayerMgr = (LayerMgr)swModel.GetLayerManager();

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Current Layer = " + swLayerMgr.GetCurrentLayer());
            Debug.Print("");

            vLayerArr = (object[])swLayerMgr.GetLayerList();
            foreach (object vLayer_loopVariable in vLayerArr)
            {
                vLayer = vLayer_loopVariable;
                swLayer = (Layer)swLayerMgr.GetLayer(vLayer.ToString());

                Debug.Print(" Layer          = " + swLayer.Name);
                Debug.Print(" Color          = " + swLayer.Color);
                Debug.Print(" Description    = " + swLayer.Description);
                Debug.Print(" ID            = " + swLayer.GetID());
                Debug.Print(" Style         = " + swLayer.Style);
                Debug.Print(" Visible       = " + swLayer.Visible);
                Debug.Print(" Width        = " + swLayer.Width);
                Debug.Print(" Printable     = " + swLayer.Printable);
            }
        }

        public SldWorks swApp;
    }
}
```

Get Layers Example (VBA)

This example shows how to get the layers in a drawing document.

```
'-----  
' Preconditions:  
' 1. Open a drawing document.  
' 2. Open an Immediate window.  
'  
' Postconditions: Inspect the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swLayerMgr As SldWorks.LayerMgr  
    Dim vLayerArr As Variant  
    Dim vLayer As Variant  
    Dim swLayer As SldWorks.Layer  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swLayerMgr = swModel.GetLayerManager  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " Current Layer = " & swLayerMgr.GetCurrentLayer  
    Debug.Print ""  
  
    vLayerArr = swLayerMgr.GetLayerList  
    For Each vLayer In vLayerArr  
        Set swLayer = swLayerMgr.GetLayer(vLayer)  
        Debug.Print " Layer = " & swLayer.Name  
        Debug.Print " Color = " & swLayer.Color  
        Debug.Print " Description = " & swLayer.Description  
        Debug.Print " ID = " & swLayer.GetID  
        Debug.Print " Style = " & swLayer.Style  
        Debug.Print " Visible = " & swLayer.Visible  
        Debug.Print " Width = " & swLayer.Width  
        Debug.Print " Printable = " & swLayer.Printable  
    Next  
End Sub
```

Get Layers Example (VB.NET)

This example shows how to get the layers in a drawing document.

```
'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()

        Dim swModel As ModelDoc2
        Dim swLayerMgr As LayerMgr
        Dim vLayerArr As Object
        Dim vLayer As Object
        Dim swLayer As Layer

        swModel = swApp.ActiveDoc
        swLayerMgr = swModel.GetLayerManager

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Current Layer = " & swLayerMgr.GetCurrentLayer)
        Debug.Print("")

        vLayerArr = swLayerMgr.GetLayerList
        For Each vLayer In vLayerArr
            swLayer = swLayerMgr.GetLayer(vLayer)

            Debug.Print(" Layer = " & swLayer.Name)
            Debug.Print(" Color = " & swLayer.Color)
            Debug.Print(" Description = " & swLayer.Description)
            Debug.Print(" ID = " & swLayer.GetID)
            Debug.Print(" Style = " & swLayer.Style)
            Debug.Print(" Visible = " & swLayer.Visible)
            Debug.Print(" Width = " & swLayer.Width)
            Debug.Print(" Printable = " & swLayer.Printable)

        Next
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Get Length of Edge Example (VBA)

This example shows how to get the length of an edge directly from the underlying curve and also from the display tessellation. The effect of the tolerance, when getting the curve tessellation, can be seen in the differences in the calculated lengths. This example exists primarily to show how to use ICurve::GetTessPts.

```

'-----
' Preconditions:
' 1. Open a part or fully resolved assembly.
' 2. Select an edge.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the length of the selected edge.
' 2. Examine the Immediate window.
'-----
Option Explicit

Public Const LINE_TYPE As Integer = 3001
Public Const CIRCLE_TYPE As Integer = 3002
Public Const ELLIPSE_TYPE As Integer = 3003
Public Const INTERSECTION_TYPE As Integer = 3004
Public Const BCURVE_TYPE As Integer = 3005
Public Const SPCURVE_TYPE As Integer = 3006
Public Const CONSTPARAM_TYPE As Integer = 3008
Public Const TRIMMED_TYPE As Integer = 3009

Function TessLength(vTessPts As Variant) As Double
    Dim i As Long

    For i = 0 To ((UBound(vTessPts) + 1) / 3 - 2)
        TessLength = TessLength + Sqr((vTessPts(3 * i + 0) - vTessPts(3 * i + 3)) ^ 2# + (vTessPts(3 * i + 1) - vTessPts(3 * i + 4)) ^ 2# +
(vTessPts(3 * i + 2) - vTessPts(3 * i + 5)) ^ 2#)
    Next i
End Function

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swEdge As SldWorks.Edge
    Dim swCurve As SldWorks.Curve
    Dim bRet As Boolean
    Dim vCurveParam As Variant
    Dim StartPt(2) As Double
    Dim EndPt(2) As Double
    Dim vStartPt As Variant
    Dim vEndPt As Variant
    Dim vTessPts As Variant

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swEdge = swSelMgr.GetSelectedObject6(1, -1)
    Set swCurve = swEdge.GetCurve
    vCurveParam = swEdge.GetCurveParams2
    StartPt(0) = vCurveParam(0)
    StartPt(1) = vCurveParam(1)
    StartPt(2) = vCurveParam(2)
    EndPt(0) = vCurveParam(3)
    EndPt(1) = vCurveParam(4)
    EndPt(2) = vCurveParam(5)
    vStartPt = StartPt
    vEndPt = EndPt

    Debug.Print "CurveType = " & swCurve.Identity
    Debug.Print "CurveLength = " & swCurve.GetLength2(vCurveParam(6), vCurveParam(7)) * 1000# & " mm"
    Debug.Print ""

    vTessPts = swCurve.GetTessPts(0.000001, 0.000001, (vStartPt), (vEndPt))
    Debug.Print "TessLength(" & UBound(vTessPts) & ") = " & TessLength(vTessPts) * 1000# & " mm"

    vTessPts = swCurve.GetTessPts(0.00001, 0.00001, (vStartPt), (vEndPt))
    Debug.Print "TessLength(" & UBound(vTessPts) & ") = " & TessLength(vTessPts) * 1000# & " mm"

    vTessPts = swCurve.GetTessPts(0.0001, 0.0001, (vStartPt), (vEndPt))
    Debug.Print "TessLength(" & UBound(vTessPts) & ") = " & TessLength(vTessPts) * 1000# & " mm"

    vTessPts = swCurve.GetTessPts(0.001, 0.001, (vStartPt), (vEndPt))
    Debug.Print "TessLength(" & UBound(vTessPts) & ") = " & TessLength(vTessPts) * 1000# & " mm"

    vTessPts = swCurve.GetTessPts(0.01, 0.01, (vStartPt), (vEndPt))
    Debug.Print "TessLength(" & UBound(vTessPts) & ") = " & TessLength(vTessPts) * 1000# & " mm"

End Sub

```

Get Length of Spline or Elliptical Edge Example (VBA)

This example shows how to get the length of an edge using display tessellation.

```
'-----
' Preconditions:
' 1. Open a part that contains a spline or elliptical edge.
' 2. Select the spline or elliptical edge.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates two 3D sketches:
'   * Control Pts contains control points for the edge.
'   * Spline Pts contains spline points for the edge.
' 2. Examine the graphics area, FeatureManager design tree,
'   the Immediate window.
'
' NOTE: A sketch with spline points is only created for a spline edge.
'-----

Option Explicit

' Tolerances that the user can set using IModeler::SetTolerances

Public Enum swTolerances_e
    swBSCurveOutputTol = 0 ' 3D bspline curve output tolerance (meters)
    swBSCurveNonRationalOutputTol = 1 ' 3D non-rational bspline curve output tolerance (meters)
    swUVCurveOutputTol = 2 ' 2D trim curve output tolerance (fraction of characteristic min. face dimension)
    swSurfChordTessellationTol = 3 ' Chord tolerance or deviation for tessellation for surfaces
    swSurfAngularTessellationTol = 4 ' Angular tolerance or deviation for tessellation for surfaces
    swCurveChordTessellationTol = 5 ' Chord tolerance or deviation for tessellation for curves
End Enum

Public Enum swCurveTypes_e
    LINE_TYPE = 3001
    CIRCLE_TYPE = 3002
    ELLIPSE_TYPE = 3003
    INTERSECTION_TYPE = 3004
    BCURVE_TYPE = 3005
    SPCURVE_TYPE = 3006
    CONSTPARAM_TYPE = 3008
    TRIMMED_TYPE = 3009
End Enum

' Define two types
Type DoubleRec
    dValue As Double
End Type

Type Long2Rec
    iLower As Long ' Assuming that a C int has 4 bytes
    iUpper As Long
End Type

' Extract two integer values out of a single double value,
' by assigning a DoubleRec to the double value and then
' copying the value over an Long2Rec and
' extracting the integer values
Function ExtractFields(ByVal dValue As Double, iLower As Integer, iUpper As Integer)
    Dim dr As DoubleRec
    Dim i2r As Long2Rec

    ' Set the double value
    dr.dValue = dValue

    ' Copy the values
    LSet i2r = dr

    ' Extract the values
    iLower = i2r.iLower
    iUpper = i2r.iUpper
End Function

Sub ProcessModellerSettings(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swModeller As SldWorks.Modeler)
    ' Process modeler settings
    Debug.Print "Modeler Settings:"
    Debug.Print "    BSCurveOutputTol          = " & swModeller.GetToleranceValue(swBSCurveOutputTol)
    Debug.Print "    BSCurveNonRationalOutputTol = " & swModeller.GetToleranceValue(swBSCurveNonRationalOutputTol)
    Debug.Print "    UVCurveOutputTol          = " & swModeller.GetToleranceValue(swUVCurveOutputTol)
    Debug.Print "    SurfChordTessellationTol   = " & swModeller.GetToleranceValue(swSurfChordTessellationTol)
    Debug.Print "    SurfAngularTessellationTol = " & swModeller.GetToleranceValue(swSurfAngularTessellationTol)
    Debug.Print "    CurveChordTessellationTol = " & swModeller.GetToleranceValue(swCurveChordTessellationTol)
    Debug.Print ""
End Sub

Sub ProcessBCurveInfo(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vBCurveParam As Variant)
    Dim nDim As Integer
    Dim nOrder As Integer
    Dim nCtrlPoints As Integer
```

```

Dim nPeriodicity As Integer
Dim nNumKnots As Integer
Dim swSketch As SldWorks.Sketch
Dim swSkFeat As SldWorks.Feature
Dim swSkPt As SldWorks.SketchPoint
Dim i As Integer

ExtractFields vBCurveParam(0), nDim, nOrder
ExtractFields vBCurveParam(1), nCtrlPoints, nPeriodicity
nNumKnots = nOrder + nCtrlPoints

Debug.Print " Dimension      = " & nDim
Debug.Print " Order          = " & nOrder
Debug.Print " Num Ctrl Pts     = " & nCtrlPoints
Debug.Print " Periodicity      = " & nPeriodicity
Debug.Print " Num Knots         = " & nNumKnots
Debug.Print ""

For i = 0 To nNumKnots - 1
    ' Knot weights should be increasing monotonically
    Debug.Print " Knot(" & i & ")      = " & vBCurveParam(2 + i)
Next i

Debug.Print ""

swModel.Insert3DSketch2 False
swModel.SetAddToDB True
swModel.SetDisplayWhenAdded False

Set swSketch = swModel.GetActiveSketch2
Set swSkFeat = swSketch

If 3 = nDim Then

    For i = 2 + nNumKnots To UBound(vBCurveParam) - 1 Step 3

        Debug.Print " Ctrl(" & (i - 2 - nNumKnots) / 3 & ")      = (" & vBCurveParam(i + 0) * 1000# & ", " & vBCurveParam(i + 1) * 1000#
        & ", " & vBCurveParam(i + 2) * 1000# & ") mm"
        Set swSkPt = swModel.CreatePoint2(vBCurveParam(i + 0), vBCurveParam(i + 1), vBCurveParam(i + 2))
        Next i
    Else
        For i = 2 + nNumKnots To UBound(vBCurveParam) - 1 Step 4
            Debug.Print " Ctrl(" & (i - 2 - nNumKnots) / 4 & ")      = (" & vBCurveParam(i + 0) * 1000# & ", " & vBCurveParam(i + 1) * 1000#
            & ", " & vBCurveParam(i + 2) * 1000# & ") mm [" & vBCurveParam(i + 3) & "]"
            Set swSkPt = swModel.CreatePoint2(vBCurveParam(i + 0), vBCurveParam(i + 1), vBCurveParam(i + 2))
            Next i
        End If

        swModel.SetDisplayWhenAdded True
        swModel.SetAddToDB False
        swModel.Insert3DSketch2 True

        swModel.ClearSelection2 True

        swSkFeat.Name = "Control Pts"
    End Sub

Sub ProcessSplineInfo(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vSplinePts As Variant)
    Dim swSketch As SldWorks.Sketch
    Dim swSkFeat As SldWorks.Feature
    Dim swSkPt As SldWorks.SketchPoint
    Dim i As Long

    swModel.Insert3DSketch2 False
    swModel.SetAddToDB True
    swModel.SetDisplayWhenAdded False

    Set swSketch = swModel.GetActiveSketch2
    Set swSkFeat = swSketch

    For i = 0 To UBound(vSplinePts) Step 3
        Debug.Print " SplinePt(" & i / 3 & ")      = (" & vSplinePts(i + 0) * 1000# & ", " & vSplinePts(i + 1) * 1000# & ", " & vSplinePts(i + 2)
        * 1000# & ") mm"
        Set swSkPt = swModel.CreatePoint2(vSplinePts(i + 0), vSplinePts(i + 1), vSplinePts(i + 2))
        Next i

        swModel.SetDisplayWhenAdded True
        swModel.SetAddToDB False
        swModel.Insert3DSketch2 True

        swModel.ClearSelection2 True

        swSkFeat.Name = "Spline Pts"
    End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModeller As SldWorks.Modeler
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swEdge As SldWorks.Edge
    Dim swCurve As SldWorks.Curve
    Dim vBCurveParam As Variant
    Dim vSplinePts As Variant
    Dim i As Long
    Dim bRet As Boolean

```

```

Set swApp = Application.SldWorks
Set swModeller = swApp.GetModeler
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swEdge = swSelMgr.GetSelectedObject5(1)
Set swCurve = swEdge.GetCurve: Debug.Assert swCurve.IsBcurve
ProcessModellerSettings swApp, swModel, swModeller
vBCurveParam = swCurve.GetBCurveParams(False)
vSplinePts = swCurve.GetSplinePts(vBCurveParam)

Debug.Print "File = " & swModel.GetPathName
Debug.Print " Identity = " & swCurve.Identity

ProcessBCurveInfo swApp, swModel, vBCurveParam

Debug.Print " "

' Do not assert because null is a valid return value
' ICurve::GetSplinePts
' * If the curve is periodic, then it must not have any repeated knots.
' * If the curve is non-periodic, it must only have repeated
' knots at its ends.
'
' The curve must be cubic or lower degree, non-rational, and
' have continuous first and second derivatives.
'
' NOTE: For a linear or quadratic curve to satisfy these continuity
' requirements, it must consist of a single segment.

If Not IsNull(vSplinePts) Then
    ProcessSplineInfo swApp, swModel, vSplinePts
    Debug.Print " "
End If

End Sub

```

Get Library Feature Data Example (C#)

This example shows how to get data about library features in model documents.

```
//-----
// Preconditions:
// 1. Open a model document that contains at least one
//    library feature.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the FeatureManager design tree searching for
//    library features.
// 2. Gets each library feature's data.
// 3. Inspect the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetReferences3_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        bool boolstatus;
        ModelDoc2 ModelDoc2;
        Feature Feature;
        Feature NextFeature;
        LibraryFeatureData LibraryFeatureData;
        int placementPlaneType;
        Object placementPlane;
        string ConfigurationName;
        String[] vConfigs;
        int x;
        bool LinkToLibraryPart;
        int LocatingDimensionsCount;
        Object vLocDimName;
        double[] vLocDimVal;
        int i;
        bool bOverrideDimVal;
        int SizeDimensionsCount;
        Object vSizDimName;
        double[] vSizDimVal;
        int ReferencesCount;
        Object vRefType;
        Object vRefName;
        Object[] vRefs;

        public void Main()
        {
            ModelDoc2 = (ModelDoc2)swApp.ActiveDoc;

            // Traverse the FeatureManager design tree searching for library features
            Feature = (Feature)ModelDoc2.FirstFeature();

            while ((Feature != null))
            {
                // Debug.Print(Feature.Name, Feature.GetTypeName());

                // If feature is library feature, then get its data; otherwise,
                // move onto the next feature in the FeatureManager design tree
                if (Feature.GetTypeName() == "LibraryFeature")
                {
                    Debug.Print("");
                    Debug.Print(Feature.Name + ", " + Feature.GetTypeName());
                    LibraryFeatureData = (LibraryFeatureData)Feature.GetDefinition();
                    boolstatus = LibraryFeatureData.AccessSelections(ModelDoc2, null);

                    // Access the selections of the library feature that define it
                    Debug.Print("LibraryFeatureData.AccessSelections = " + boolstatus.ToString());

                    // Get its placement type
                    placementPlane =
                    LibraryFeatureData.GetPlacementPlane2((int)swLibFeatureData_e.swLibFeatureData_PartRespect, out placementPlaneType);
                    Debug.Print("PlacementPlaneType = " + placementPlaneType.ToString());

                    // Get the name of the active library feature configuration
                    ConfigurationName = LibraryFeatureData.ConfigurationName;

                    Debug.Print("ConfigurationName = " + ConfigurationName);

                    // Determine if the library feature is linked to
                    // the library feature part
                }
            }
        }
    }
}
```

```

LinkToLibraryPart = LibraryFeatureData.LinkToLibraryPart;
Debug.Print("LinkToLibraryPart = " + LinkToLibraryPart.ToString());

// If the library feature part document is open, or
// if the library feature is linked to the library feature
// part document, then get the names of all of the
// library feature configurations; if it's not open,
// then only the name of the active library configuration
// is returned
vConfigs = (String[])LibraryFeatureData.GetAllConfigurationNames();
Debug.Print("Configuration names :");

if ((vConfigs != null))
{
    for (x = vConfigs.GetLowerBound(0); x <= vConfigs.GetUpperBound(0); x++)
    {
        Debug.Print(" " + vConfigs[x]);
    }
}

// Get the number of locating dimensions
LocatingDimensionsCount = LibraryFeatureData.GetDimensionsCount(0);
Debug.Print("LocatingDimensionsCount = " + LocatingDimensionsCount.ToString());

// Get the locating dimensions
vLocDimVal = (double[])LibraryFeatureData.GetDimensions(0, out vLocDimName);
Debug.Print("Locating dimensions :");

if ((vLocDimName != null))
{
    String[] dimNames = (String[])vLocDimName;
    for (i = dimNames.GetLowerBound(0); i <= dimNames.GetUpperBound(0); i++)
    {
        Debug.Print(" " + dimNames[i], vLocDimVal[i]);
    }
}

// Determine if existing dimension values of the library
// feature can be overridden
bOverrideDimVal = LibraryFeatureData.OverrideDimension;
Debug.Print("OverrideDimension = " + bOverrideDimVal.ToString());

// Get the number of feature dimensions
SizeDimensionsCount = LibraryFeatureData.GetDimensionsCount(1);
Debug.Print("SizeDimensionsCount = " + SizeDimensionsCount.ToString());

// Get the feature dimensions
vSizDimVal = (double[])LibraryFeatureData.GetDimensions(1, out vSizDimName);
Debug.Print("Size dimensions :");

if ((vSizDimName != null))
{
    String[] sizDimNames = (String[])vSizDimName;
    for (i = sizDimNames.GetLowerBound(0); i <= sizDimNames.GetUpperBound(0); i++)
    {
        Debug.Print(" " + sizDimNames[i], vSizDimVal[i]);
    }
}

// Get the number of references
ReferencesCount = LibraryFeatureData.GetReferencesCount();
Debug.Print("GetReferencesCount = " + ReferencesCount.ToString());

// Get the references
vRefs = (object[])LibraryFeatureData.GetReferences3((int)swLibFeatureData_e.swLibFeatureData_PartRespect,
out vRefType, out vRefName);
if ((vRefType != null))
{
    int[] refTypes = (int[])vRefType;
    string[] refNames = (string[])vRefName;
    Debug.Print("Reference types and names: ");
    for (i = refTypes.GetLowerBound(0); i <= refTypes.GetUpperBound(0); i++)
    {
        Debug.Print(" " + refTypes[i].ToString() + ", " + (string)refNames[i]);
    }
}

//Release the selections that define the library feature
LibraryFeatureData.ReleaseSelectionAccess();
}

// Get the next feature until there are no more
NextFeature = (Feature)Feature.GetNextFeature();
Feature = null;
Feature = NextFeature;
NextFeature = null;
}
}

public Sldworks swApp;

```


Get Library Feature Data Example (VBA)

This example shows how to get data about library features in model documents.

```

'-----
' Preconditions:
' 1. Open a model document that contains at least one
'    library feature.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the FeatureManager design tree searching for
'    library features.
' 2. Gets each library feature's data.
' 3. Inspect the Immediate window.
'-----
Option Explicit

Dim boolstatus As Boolean
Dim swApp As SldWorks.SldWorks
Dim ModelDoc2 As SldWorks.ModelDoc2
Dim Feature As SldWorks.Feature
Dim NextFeature As SldWorks.Feature
Dim LibraryFeatureData As LibraryFeatureData
Dim placementPlaneType As Long
Dim placementPlane As Object
Dim ConfigurationName As String
Dim vConfigs As Variant
Dim vConfigName As Variant
Dim x As Long
Dim LinkToLibraryPart As Boolean
Dim LocatingDimensionsCount As Long
Dim vLocDimName As Variant
Dim vLocDimVal As Variant
Dim i As Long
Dim bOverrideDimVal As Boolean
Dim SizeDimensionsCount As Long
Dim vSizDimName As Variant
Dim vSizDimVal As Variant
Dim ReferencesCount As Long
Dim vRefType As Variant
Dim vRefName As Variant
Dim vRefs As Variant

Sub main()

Set swApp = Application.SldWorks
Set ModelDoc2 = swApp.ActiveDoc

' Traverse the FeatureManager design tree searching for library features
Set Feature = ModelDoc2.FirstFeature

While Not Feature Is Nothing

    'Debug.Print Feature.Name, Feature.GetTypeName

    ' If feature is a library feature, then get its data; otherwise,
    ' move onto the next feature in the FeatureManager design tree

    If Feature.GetTypeName = "LibraryFeature" Then
        Debug.Print ""
        Debug.Print Feature.Name, Feature.GetTypeName
        Set LibraryFeatureData = Feature.GetDefinition
        boolstatus = LibraryFeatureData.AccessSelections(ModelDoc2, Nothing)

        ' Access the selections of the library feature that define it
        Debug.Print "LibraryFeatureData.AccessSelections = " & boolstatus

        ' Get its placement type
        Set placementPlane = LibraryFeatureData.GetPlacementPlane2(swLibFeatureData_e.swLibFeatureData_PartRespect, placementPlaneType)
        Debug.Print "PlacementPlaneType = " & placementPlaneType
        placementPlane.Select False

        ' Get the name of the active library feature configuration
        ConfigurationName = LibraryFeatureData.ConfigurationName
        Debug.Print "ConfigurationName = " & ConfigurationName

        ' Determine if the library feature is linked to
        ' the library feature part
        LinkToLibraryPart = LibraryFeatureData.LinkToLibraryPart
        Debug.Print "LinkToLibraryPart = " & LinkToLibraryPart

        ' If the library feature part document is open or
        ' if the library feature is linked to the library feature
        ' part document, then get the names of all of the
        ' library feature configurations; if neither,
        ' then only the name of the active library configuration
        ' is returned
        vConfigs = LibraryFeatureData.GetAllConfigurationNames
        Debug.Print "Configuration names :"
```

```

If Not IsEmpty(vConfigs) Then
    For x = LBound(vConfigs) To UBound(vConfigs)
        Debug.Print " " & vConfigs(x)
    Next x
End If

' Get the number of locating dimensions
LocatingDimensionsCount = LibraryFeatureData.GetDimensionsCount(0)
Debug.Print "LocatingDimensionsCount = " & LocatingDimensionsCount

' Get the locating dimensions
vLocDimVal = LibraryFeatureData.GetDimensions(0, vLocDimName)
Debug.Print "Locating dimensions : "

If Not IsEmpty(vLocDimName) Then
    For i = LBound(vLocDimName) To UBound(vLocDimName)
        Debug.Print " " & vLocDimName(i), vLocDimVal(i)
    Next i
End If

' Determine if existing dimension values of the library
' feature can be overridden
bOverrideDimVal = LibraryFeatureData.OverrideDimension
Debug.Print "OverrideDimension = " & bOverrideDimVal

' Get the number of feature dimensions
SizeDimensionsCount = LibraryFeatureData.GetDimensionsCount(1)
Debug.Print "SizeDimensionsCount = " & SizeDimensionsCount

' Get the feature dimensions
vSizDimVal = LibraryFeatureData.GetDimensions(1, vSizDimName)
Debug.Print "Size dimensions : "
If Not IsEmpty(vSizDimName) Then
    For i = LBound(vSizDimName) To UBound(vSizDimName)
        Debug.Print " " & vSizDimName(i), vSizDimVal(i)
    Next i
End If

' Get the number of references
ReferencesCount = LibraryFeatureData.GetReferencesCount
Debug.Print "GetReferencesCount = " & ReferencesCount

' Get the references
vRefs = LibraryFeatureData.GetReferences3(swLibFeatureData_e.swLibFeatureData_PartRespect, vRefType, vRefName)
If Not IsEmpty(vRefType) Then
    Debug.Print "Reference types and names: "
    For i = LBound(vRefType) To UBound(vRefType)
        Debug.Print " " & vRefType(i) & ", " & vRefName(i)
        vRefs(i).Select False
    Next i
End If

'Release the selections that define the library feature
LibraryFeatureData.ReleaseSelectionAccess

End If

' Get the next feature until there are no more
Set NextFeature = Feature.GetNextFeature
Set Feature = Nothing
Set Feature = NextFeature
Set NextFeature = Nothing

Wend

End Sub

```

Get Library Feature Data Example (VB.NET)

This example shows how to get data about library features in model documents.

```

'-----
' Preconditions:
' 1. Open a model document that contains at least one
'    library feature.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the FeatureManager design tree searching for
'    library features.
' 2. Gets each library feature's data.
' 3. Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim boolstatus As Boolean
    Dim ModelDoc2 As ModelDoc2
    Dim Feature As Feature
    Dim NextFeature As Feature
    Dim LibraryFeatureData As LibraryFeatureData
    Dim placementPlaneType As Integer
    Dim placementPlane As Object
    Dim ConfigurationName As String
    Dim vConfigs As Object
    Dim vConfigName As Object
    Dim x As Integer
    Dim LinkToLibraryPart As Boolean
    Dim LocatingDimensionsCount As Integer
    Dim vLocDimName As Object
    Dim vLocDimVal As Object
    Dim i As Integer
    Dim bOverrideDimVal As Boolean
    Dim SizeDimensionsCount As Integer
    Dim vSizDimName As Object
    Dim vSizDimVal As Object
    Dim ReferencesCount As Integer
    Dim vRefType As Object
    Dim vRefName As Object
    Dim vRefs As Object

    Sub main()

        ModelDoc2 = swApp.ActiveDoc

        ' Traverse the FeatureManager design tree searching for library features
        Feature = ModelDoc2.FirstFeature

        While Not Feature Is Nothing

            'Debug.Print(Feature.Name, Feature.GetTypeName)

            ' If feature is library feature, then get its data; otherwise,
            ' move onto the next feature in the FeatureManager design tree
            If Feature.GetTypeName = "LibraryFeature" Then

                Debug.Print ""
                Debug.Print(Feature.Name, Feature.GetTypeName)
                LibraryFeatureData = Feature.GetDefinition
                boolstatus = LibraryFeatureData.AccessSelections(ModelDoc2, Nothing)

                ' Access the selections of the library feature that define it
                Debug.Print("LibraryFeatureData.AccessSelections = " & boolstatus)

                ' Get its placement type
                placementPlane = LibraryFeatureData.GetPlacementPlane2(swLibFeatureData_e.swLibFeatureData_PartRespect,
placementPlaneType)
                Debug.Print("PlacementPlaneType = " & placementPlaneType)
                placementPlane.Select(False)

                ' Get the name of the active library feature configuration
                ConfigurationName = LibraryFeatureData.ConfigurationName
                Debug.Print("ConfigurationName = " & ConfigurationName)

                ' Determine if the library feature is linked to
                ' the library feature part
            
```

```

LinkToLibraryPart = LibraryFeatureData.LinkToLibraryPart
Debug.Print("LinkToLibraryPart = " & LinkToLibraryPart)

' If the library feature part document is open, or
' if the library feature is linked to the library feature
' part document, then get the names of all of the
' library feature configurations; if it's not open,
' then only the name of the active library configuration
' is returned
vConfigs = LibraryFeatureData.GetAllConfigurationNames
Debug.Print("Configuration names :")

If Not vConfigs Is Nothing Then
    For x = LBound(vConfigs) To UBound(vConfigs)
        Debug.Print(" " & vConfigs(x))
    Next x
End If

' Get the number of locating dimensions
LocatingDimensionsCount = LibraryFeatureData.GetDimensionsCount(0)
Debug.Print("LocatingDimensionsCount = " & LocatingDimensionsCount)

' Get the locating dimensions
vLocDimVal = LibraryFeatureData.GetDimensions(0, vLocDimName)
Debug.Print("Locating dimensions :")

If Not vLocDimName Is Nothing Then
    For i = LBound(vLocDimName) To UBound(vLocDimName)
        Debug.Print(" " & vLocDimName(i), vLocDimVal(i))
    Next i
End If

' Determine if existing dimension values of the library
' feature can be overridden
bOverrideDimVal = LibraryFeatureData.OverrideDimension
Debug.Print("OverrideDimension = " & bOverrideDimVal)

' Get the number of feature dimensions
SizeDimensionsCount = LibraryFeatureData.GetDimensionsCount(1)
Debug.Print("SizeDimensionsCount = " & SizeDimensionsCount)

' Get the feature dimensions
vSizDimVal = LibraryFeatureData.GetDimensions(1, vSizDimName)
Debug.Print("Size dimensions :")
If Not vSizDimName Is Nothing Then
    For i = LBound(vSizDimName) To UBound(vSizDimName)
        Debug.Print(" " & vSizDimName(i), vSizDimVal(i))
    Next i
End If

' Get the number of references
ReferencesCount = LibraryFeatureData.GetReferencesCount
Debug.Print("GetReferencesCount = " & ReferencesCount)

' Get the references
vRefs = LibraryFeatureData.GetReferences3(swLibFeatureData_e.swLibFeatureData_PartRespect, vRefType,
vRefName)
If Not vRefType Is Nothing Then
    Debug.Print("Reference types and names: ")
    For i = LBound(vRefType) To UBound(vRefType)
        Debug.Print(" " & vRefType(i) & ", " & vRefName(i))
        vRefs(i).Select(False)
    Next i
End If

'Release the selections that define the library feature
LibraryFeatureData.ReleaseSelectionAccess()

End If

' Get the next feature until there are no more
NextFeature = Feature.GetNextFeature
Feature = Nothing
Feature = NextFeature
NextFeature = Nothing

End While

End Sub

Public swApp As SldWorks

End Class

```

Get License Type of SOLIDWORKS Design Example (C#)

This example shows how to get the type of license used when the model was created.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part document.
// 2. Gets the type of license used when
//    the model was created.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            int licenseType = 0;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", re
f errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            licenseType = swModelDocExt.GetLicenseType();
            Debug.Print("Type of license used when the model was created as defined by swLicenseType_e: " + licenseType);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get License Type of SOLIDWORKS Design Example (VBA)

This example shows how to get the type of license used when the model was created.

```
'-----  
' Preconditions:  
' 1. Verify that the part to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the part document.  
' 2. Gets the type of license used when  
'    the model was created.  
' 3. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim licenseType As Long  
    Dim errors As Long  
    Dim warnings As Long  
    Dim fileName As String  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
  
    licenseType = swModelDocExt.GetLicenseType  
    Debug.Print "Type of license used when the model was created as defined by swLicenseType_e: " & licenseType  
  
End Sub
```

Get License Type of SOLIDWORKS Design Example (VB.NET)

This example shows how to get the type of license used when the model was created.

```
-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part document.
' 2. Gets the type of license used when
'    the model was created.
' 3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim licenseType As Integer
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        licenseType = swModelDocExt.GetLicenseType
        Debug.Print("Type of license used when the model was created as defined by swLicenseType_e: " & licenseType)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Line Parameters Example (VBA)

This example shows how to get a line's parameters.

```
'-----  
' Preconditions:  
' 1. Open a model document and select an edge.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the parameters of the curve, which  
'    is a line.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swEdge As SldWorks.Edge  
    Dim swCurve As SldWorks.Curve  
    Dim vLineParam As Variant  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    Set swSelMgr = swModel.SelectionManager  
    Set swEdge = swSelMgr.GetSelectedObject6(1, -1)  
    Set swCurve = swEdge.GetCurve  
    vLineParam = swCurve.LineParams  
    Debug.Print "Root point = (" & vLineParam(0) * 1000# & ", " & vLineParam(1) * 1000# & ", " & vLineParam(2) * 1000# & ") mm"  
    Debug.Print "Direction = (" & vLineParam(3) & ", " & vLineParam(4) & ", " & vLineParam(5) & ")"  
  
End Sub
```

Get Linear Pattern Feature Data Example (C#)

This example shows how to get data for a linear pattern feature.

```
//-----
// Preconditions:
// 1. Open a part document with a LPattern1 feature.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets, sets, and prints linear pattern feature data to
//    the Immediate window.
// 2. Examine the FeatureManager design tree, graphics area, and Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace LinearPatternFeaturDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            LinearPatternFeatureData swLinearPatternFeatureData = default(LinearPatternFeatureData);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get the linear pattern feature data
            status = swModelDocExt.SelectByID("LPattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swLinearPatternFeatureData = (LinearPatternFeatureData)swFeature.GetDefinition();
            swLinearPatternFeatureData.AccessSelections(swModel, null);
            Debug.Print("Number of features in pattern: " + swLinearPatternFeatureData.GetPatternFeatureCount());
            Debug.Print("Total number of instances: ");
            Debug.Print(" Direction 1: " + swLinearPatternFeatureData.D1TotalInstances);
            Debug.Print(" Direction 2: " + swLinearPatternFeatureData.D2TotalInstances);
            //Change Direction 2 spacing
            Debug.Print("Direction 2 spacing: ");
            Debug.Print(" Original: " + swLinearPatternFeatureData.D2Spacing);
            swLinearPatternFeatureData.D2Spacing = 0.04;
            Debug.Print(" Updated: " + swLinearPatternFeatureData.D2Spacing);
            swFeature.ModifyDefinition(swLinearPatternFeatureData, swModel, null);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Linear Pattern Feature Data Example (VBA)

This example shows how to get data for a linear pattern feature.

```
'-----
' Preconditions:
' 1. Open a part document with a LPattern1 feature.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets, sets, and prints linear pattern feature data to
'    the Immediate window.
' 2. Examine the FeatureManager design tree, graphics area, and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swLinearPatternFeatureData As SldWorks.LinearPatternFeatureData
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    'Get the linear pattern feature data
    status = swModelDocExt.SelectByID("LPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swLinearPatternFeatureData = swFeature.GetDefinition
    swLinearPatternFeatureData.AccessSelections swModel, Nothing
    Debug.Print "Number of features in pattern: " & swLinearPatternFeatureData.GetPatternFeatureCount
    Debug.Print "Total number of instances: "
    Debug.Print " Direction 1: " & swLinearPatternFeatureData.D1TotalInstances
    Debug.Print " Direction 2: " & swLinearPatternFeatureData.D2TotalInstances
    'Change Direction 2 spacing
    Debug.Print "Direction 2 spacing: "
    Debug.Print " Original: " & swLinearPatternFeatureData.D2Spacing
    swLinearPatternFeatureData.D2Spacing = 0.04
    Debug.Print " Updated: " & swLinearPatternFeatureData.D2Spacing
    swFeature.ModifyDefinition swLinearPatternFeatureData, swModel, Nothing

End Sub
```

Get Linear Pattern Feature Data Example (VB.NET)

This example shows how to get data for a linear pattern feature.

```

'-----
' Preconditions:
' 1. Open a part document with a LPattern1 feature.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets, sets, and prints linear pattern feature data to
'    the Immediate window.
' 2. Examine the FeatureManager design tree, graphics area, and Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swSelectionMgr As SelectionMgr
        Dim swLinearPatternFeatureData As LinearPatternFeatureData
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Get the linear pattern feature data
        status = swModelDocExt.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swLinearPatternFeatureData = swFeature.GetDefinition
        swLinearPatternFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Number of features in pattern: " & swLinearPatternFeatureData.GetPatternFeatureCount)
        Debug.Print("Total number of instances: ")
        Debug.Print(" Direction 1: " & swLinearPatternFeatureData.D1TotalInstances)
        Debug.Print(" Direction 2: " & swLinearPatternFeatureData.D2TotalInstances)
        'Change Direction 2 spacing
        Debug.Print("Direction 2 spacing: ")
        Debug.Print(" Original: " & swLinearPatternFeatureData.D2Spacing)
        swLinearPatternFeatureData.D2Spacing = 0.04
        Debug.Print(" Updated: " & swLinearPatternFeatureData.D2Spacing)
        swFeature.ModifyDefinition(swLinearPatternFeatureData, swModel, Nothing)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Lines in Sketch Example (VBA)

This example shows how to get the number of lines in a selected sketch and their start and end point coordinates.

```
'-----  
' Preconditions:  
' 1. Open a part and select a sketch.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the number of lines in the selected  
' sketch and their start and end point  
' coordinates.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
    Dim swSketch As SldWorks.Sketch  
    Dim NumLines As Long  
    Dim vLines As Variant  
    Dim i As Variant  
    Dim bRet As Boolean  
    Dim TrueFalse As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swSelMgr = swModel.SelectionManager  
  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Debug.Print "Feature = " & swFeat.GetTypeName  
    Set swSketch = swFeat.GetSpecificFeature2  
    NumLines = swSketch.GetLineCount2(1) ' Exclude crosshatch lines  
    Debug.Print " Number of lines = " & NumLines  
    vLines = swSketch.GetLines2(1) 'Exclude crosshatch lines  
    For i = 0 To NumLines - 1  
        Debug.Print " Line(" & i & ")"  
        Debug.Print " Start = (" & vLines(12 * i + 6) * 1000# & "," & vLines(12 * i + 7) * 1000# & "," & vLines(12 * i + 8) * 1000# & ") mm"  
        Debug.Print " End = (" & vLines(12 * i + 9) * 1000# & "," & vLines(12 * i + 10) * 1000# & "," & vLines(12 * i + 11) * 1000# & ") mm"  
    Next i  
  
End Sub
```

Get Linked Dimensions Example (VBA)

This example shows how to get the linked dimensions in a part document.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\lesson2\tutor1.sldprt.
' 2. Right-click Annotations and click Show Feature Dimensions.
' 3. Right-click either 120 dimension and click Link Values.
' 4. Type AreaDimension and click OK.
' 5. Right-click the other 120 dimension and click Link Values.
' 6. Type AreaDimension and click OK.
' 7. Open the Immediate window.
'
' Postconditions:
' 1. Examine the Immediate window.
' 2. Click either 120 dimension, change 120 to 150, and press Enter.
' 3. Changes the other 120 dimension to 150 because the dimensions
'    are linked.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Private Type DimLink
    Name As String
    DimName() As String
End Type

Private Sub ProcessDimLinkList(ThisDimLinkList() As DimLink)
    Dim i As Long
    Dim j As Long

    For i = 0 To UBound(ThisDimLinkList)
        Debug.Print ThisDimLinkList(i).Name
        For j = 0 To UBound(ThisDimLinkList(i).DimName)
            Debug.Print " " & ThisDimLinkList(i).DimName(j)
        Next j
    Next i
End Sub

Private Sub AddToDimLinkList(ThisDimLink As DimLink, ThisDimLinkList() As DimLink)
    Debug.Assert UBound(ThisDimLink.DimName) = 0
    Dim i As Long

    For i = 0 To UBound(ThisDimLinkList)
        If ThisDimLinkList(i).Name = ThisDimLink.Name Then
            ReDim Preserve ThisDimLinkList(i).DimName(UBound(ThisDimLinkList(i).DimName) + 1)
            ThisDimLinkList(i).DimName(UBound(ThisDimLinkList(i).DimName)) = ThisDimLink.DimName(0)
            Exit Sub
        End If
    Next i

    If 0 = UBound(ThisDimLinkList) And "" = ThisDimLinkList(0).Name Then
        ThisDimLinkList(0).Name = ThisDimLink.Name
        ThisDimLinkList(0).DimName(UBound(ThisDimLinkList(0).DimName)) = ThisDimLink.DimName(0)
        Exit Sub
    End If

    ReDim Preserve ThisDimLinkList(UBound(ThisDimLinkList) + 1)
    ReDim Preserve ThisDimLinkList(UBound(ThisDimLinkList)).DimName(0)

    ThisDimLinkList(UBound(ThisDimLinkList)).Name = ThisDimLink.Name
    ThisDimLinkList(UBound(ThisDimLinkList)).DimName(UBound(ThisDimLinkList).DimName)) = ThisDimLink.DimName(0)
End Sub

Private Function IsInDimLinkList(ThisDimLink As DimLink, ThisDimLinkList() As DimLink) As Boolean
    Dim i As Long
    Dim j As Long

    For i = 0 To UBound(ThisDimLinkList)
        If ThisDimLinkList(i).Name = ThisDimLink.Name Then
            For j = 0 To UBound(ThisDimLinkList(i).DimName)
                If ThisDimLinkList(i).DimName(j) = ThisDimLink.DimName(0) Then
                    IsInDimLinkList = True
                    Exit Function
                End If
            Next j
        End If
    Next i
End Function

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeat As SldWorks.Feature
    Dim swDimen As SldWorks.Dimension
    Dim swDispDim As SldWorks.DisplayDimension
    Dim swDispDimAnn As SldWorks.Annotation
    Dim OneDimLink As DimLink

```

```

Dim DimLinkList() As DimLink
Dim bRet As Boolean

ReDim DimLinkList(0)
ReDim DimLinkList(0).DimName(0)
ReDim OneDimLink.DimName(0)

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swFeat = swModel.FirstFeature

Debug.Print "File = " & swModel.GetPathName

Do While Not swFeat Is Nothing
    Set swDispDim = swFeat.GetFirstDisplayDimension
    If Not swDispDim Is Nothing Then
        Debug.Print " " & swFeat.Name
    End If
    Do While Not swDispDim Is Nothing
        Set swDispDimAnn = swDispDim.GetAnnotation
        Set swDimen = swDispDim.GetDimension
        If swDispDim.IsLinked Then
            OneDimLink.Name = swDispDim.GetLinkedText
            OneDimLink.DimName(0) = swDimen.FullName
            Debug.Print " " & swDispDimAnn.GetName & " [" & swDimen.FullName & "] -- > " & swDispDim.GetLinkedText

            If Not IsInDimLinkList(OneDimLink, DimLinkList) Then
                AddToDimLinkList OneDimLink, DimLinkList
            End If
        End If
        Set swDispDim = swFeat.GetNextDisplayDimension(swDispDim)
    Loop
    Set swFeat = swFeat.GetNextFeature
Loop

ProcessDimLinkList DimLinkList

End Sub

```

Get List of Configurations Example (C#)

This example shows how to get a list of configuration names for the active document.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Gets the name of the file, name of the active configuration,
//    and the names of all configurations.
// 3. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            string[] configNames = null;
            string configName = null;
            Configuration swConfig = default(Configuration);
            int i = 0;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\smartcomponents\\pillow_block.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, ""
, ref errors, ref warnings);

            //Get and print active model path and name
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("");
            //Get and print name of active configuration
            swConfig = (Configuration)swModel.GetActiveConfiguration();
            Debug.Print(" Name of active configuration = " + swConfig.Name);
            Debug.Print("");

            //Get and print names of all configurations
            configNames = (string[])swModel.GetConfigurationNames();
            for (i = 0; i < configNames.Length; i++)
            {
                configName = (string)configNames[i];
                swConfig = (Configuration)swModel.GetConfigurationByName(configName);
                Debug.Print(" Name of configuration(" + i + ") = " + configName);
                Debug.Print(" Use alternate name in BOM = " + swConfig.UseAlternateNameInBOM);
                Debug.Print(" Alternate name = " + swConfig.AlternateName);
                Debug.Print(" Comment = " + swConfig.Comment);
                Debug.Print("");
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get List of Configurations Example (VBA)

This example shows how to get a list of configuration names for the active document.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Gets the name of the file, name of the active configuration,
'    and the names of all configurations.
' 3. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim configNames() As String
    Dim configName As String
    Dim swConfig As SldWorks.Configuration
    Dim i As Long
    Dim errors As Long
    Dim warnings As Long
    Dim fileName As String

    Set swApp = SolidWorks.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\pillow_block.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Get and print active model path and name
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print ""
    'Get and print name of active configuration
    Set swConfig = swModel.GetActiveConfiguration
    Debug.Print " Name of active configuration = " & swConfig.Name
    Debug.Print ""

    'Get and print names of all configurations
    configNames = swModel.GetConfigurationNames
    For i = 0 To UBound(configNames)
        configName = configNames(i)
        Set swConfig = swModel.GetConfigurationByName(configName)
        Debug.Print " Name of configuration(" & i & ") = " & configName
        Debug.Print " Use alternate name in BOM = " & swConfig.UseAlternateNameInBOM
        Debug.Print " Alternate name = " & swConfig.AlternateName
        Debug.Print " Comment = " & swConfig.Comment
        Debug.Print ""
    Next i

End Sub
```

Get List of Configurations Example (VB.NET)

This example shows how to get a list of configuration names for the active document.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Gets the name of the file, name of the active configuration,
'    and the names of all configurations.
' 3. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim configNames() As String
        Dim configName As String
        Dim swConfig As Configuration
        Dim i As Integer
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\pillow_block.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Get and print active model path and name
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("")
        'Get and print name of active configuration
        swConfig = swModel.GetActiveConfiguration
        Debug.Print(" Name of active configuration = " & swConfig.Name)
        Debug.Print("")

        'Get and print names of all configurations
        configNames = swModel.GetConfigurationNames
        For i = 0 To UBound(configNames)
            configName = configNames(i)
            swConfig = swModel.GetConfigurationByName(configName)
            Debug.Print(" Name of configuration(" & i & ") = " & configName)
            Debug.Print(" Use alternate name in BOM = " & swConfig.UseAlternateNameInBOM)
            Debug.Print(" Alternate name = " & swConfig.AlternateName)
            Debug.Print(" Comment = " & swConfig.Comment)
            Debug.Print("")
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get List of Open Documents Example (C++ COM)

This example shows how to get a list of the currently open SOLIDWORKS Design documents.

```
// -----
void APITestFunction()
{
    HRESULT          hr = S_OK;
    VARIANT_BOOL      bRet = VARIANT_FALSE;
    long              nRetVal = -1;
    CComPtr<ISldWorks> pSldWorks;
    CComPtr<IEnumDocuments2> pEnumDoc;
    long              nFetched = -1;
    CComPtr<IModelDoc2> pModelDoc;
    long              i = -1;
    // Connect to SOLIDWORKS Design application
    pSldWorks = TheApplication->GetSWApp();
    ASSERT(pSldWorks);
    hr = pSldWorks->EnumDocuments2(&pEnumDoc);
    ASSERT(pEnumDoc);
    hr = pEnumDoc->Reset();
    do
    {
        CComBSTR      sPathName;
        // Reset before reuse
        pModelDoc = NULL;
        // Could also check HRESULT or nFetched
        hr = pEnumDoc->Next(1, &pModelDoc, &nFetched);
        if (pModelDoc)
        {
            // Debugging only
            hr = pModelDoc->GetPathName(&sPathName);
        }
    } while (pModelDoc);
    return;
} //void APITestFunction()
// -----
```

Get Loaded Sheets Example (C#)

This example shows how to determine which sheets in a drawing are loaded.

```
//-----
// Preconditions:
// 1. Click File > Open.
// 2. Browse to public_documents\samples\tutorial\advdrawings.
// 3. Select foodprocessor.slddrw.
// 4. Click Select sheets to open > Selected > Sheet1* (Load) > OK > Open.
// 5. Open the Immediate window.
//
// Postconditions:
// 1. Loads Sheet1 only.
// 2. Mouse over the Sheet2, Sheet3, and Sheet4 tabs and
//    examine the Immediate window to verify step 1.
//
// NOTE: Because this drawing is used elsewhere, do not save
// changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace IsLoadedSheetCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            object[] vSheetName = null;
            int i = 0;
            bool bRet = false;
            string sheetName;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;

            // Get the sheets in the drawing document
            vSheetName = (object[])swDraw.GetSheetNames();

            // Traverse the sheets and determine whether
            // they're loaded
            for (i = 0; i < vSheetName.Length; i++)
            {
                sheetName = (string)vSheetName[i];
                bRet = swDraw.ActivateSheet(sheetName);
                Sheet swSheet = default(Sheet);
                swSheet = (Sheet)swDraw.GetCurrentSheet();
                if ((swSheet.IsLoaded()))
                {
                    Debug.Print(vSheetName[i] + " is loaded.");
                }
                else
                {
                    Debug.Print(vSheetName[i] + " is not loaded.");
                }
            }
        }
    }
}
```

```
}  
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}  
}
```

Get Loaded Sheets Example (VBA)

This example shows how to determine which sheets in a drawing document are loaded.

```
'-----
' Preconditions:
' 1. Click File > Open.
' 2. Browse to public_documents\samples\tutorial\advdrawings.
' 3. Select foodprocessor.slddrw.
' 4. Click Select sheets to open > Selected > Sheet1* (Load) > OK > Open.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Loads Sheet1 only.
' 2. Mouse over the Sheet2, Sheet3, and Sheet4 tabs and
'    examine the Immediate window to verify step 1.
'
' NOTE: Because this drawing is used elsewhere, do not save
' changes.
'-----
```

```
Sub main()
```

```
    Dim swApp                As SldWorks.SldWorks
    Dim swModel               As SldWorks.ModelDoc2
    Dim swDraw                As SldWorks.DrawingDoc
    Dim vSheetName            As Variant
    Dim nRetVal               As Long
    Dim i                     As Long
    Dim bRet                  As Boolean
```

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
```

```
    ' Get the sheets in the drawing document
    vSheetName = swDraw.GetSheetNames
```

```
    ' Traverse the drawing sheets and determine whether
    ' they're loaded
    For i = 0 To UBound(vSheetName)
        bRet = swDraw.ActivateSheet(vSheetName(i))
        Dim swSheet As Sheet
        Set swSheet = swDraw.GetCurrentSheet
        If (swSheet.IsLoaded) Then
            Debug.Print vSheetName(i) & " is loaded."
        Else
            Debug.Print vSheetName(i) & " is not loaded."
        End If
    Next i
End Sub
```

Get Loaded Sheets Example (VB.NET)

This example shows how to determine if the sheets in a drawing are loaded.

```
'-----
' Preconditions:
' 1. Click File > Open.
' 2. Browse to public_documents\samples\tutorial\advdrawings.
' 3. Select foodprocessor.slddrw.
' 4. Click Select sheets to open > Selected > Sheet1* (Load) > OK > Open.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Loads Sheet1 only.
' 2. Mouse over the Sheet2, Sheet3, and Sheet4 tabs and
'    examine the Immediate window to verify step 1.
'
' NOTE: Because this drawing is used elsewhere, do not save
' changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swDraw As DrawingDoc
```

```
        Dim vSheetName As Object
```

```
        Dim i As Integer
```

```
        Dim bRet As Boolean
```

```
        swModel = swApp.ActiveDoc
```

```
        swDraw = swModel
```

```
        ' Get the sheets in the drawing document
```

```
        vSheetName = swDraw.GetSheetNames
```

```
        ' Traverse the drawing sheets and determine whether
        ' they're loaded
```

```
        For i = 0 To UBound(vSheetName)
```

```
            bRet = swDraw.ActivateSheet(vSheetName(i))
```

```
            Dim swSheet As Sheet
```

```
            swSheet = swDraw.GetCurrentSheet
```

```
            If (swSheet.IsLoaded) Then
```

```
                Debug.Print(vSheetName(i) & " is loaded.")
```

```
            Else
```

```
                Debug.Print(vSheetName(i) & " is not loaded.")
```

```
            End If
```

```
        Next i
```

```
    End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Get Locations and Names of Document Templates Example (VBA)

This example shows how to get the locations and names of SOLIDWORKS Design document templates.

```
'-----  
' Preconditions: Open the Immediate window.  
' Postconditions: Examine the Immediate window.  
'-----  
  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Set swApp = Application.SldWorks  
  
    Debug.Print "FileLocationsDocumentTemplates = " & swApp.GetUserPreferenceStringValue(swFileLocationsDocumentTemplates)  
    Debug.Print "FileLocationsSheetFormat      = " & swApp.GetUserPreferenceStringValue(swFileLocationsSheetFormat)  
    Debug.Print "DefaultTemplatePart           = " & swApp.GetUserPreferenceStringValue(swDefaultTemplatePart)  
    Debug.Print "DefaultTemplateAssembly       = " & swApp.GetUserPreferenceStringValue(swDefaultTemplateAssembly)  
    Debug.Print "DefaultTemplateDrawing        = " & swApp.GetUserPreferenceStringValue(swDefaultTemplateDrawing)  
    Debug.Print ""  
  
    Debug.Print "Draw template = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperASize, 0#, 0#)  
    Debug.Print "Part template = " & swApp.GetDocumentTemplate(swDocPART, "", 0, 0#, 0#)  
    Debug.Print "Assy template = " & swApp.GetDocumentTemplate(swDocASSEMBLY, "", 0, 0#, 0#)  
    Debug.Print ""  
  
    Debug.Print "Draw templates:"  
    Debug.Print "  A   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperASize, 0#, 0#)  
    Debug.Print "  A vert = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperASizeVertical, 0#, 0#)  
    Debug.Print "  B   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperBSize, 0#, 0#)  
    Debug.Print "  C   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperCSize, 0#, 0#)  
    Debug.Print "  D   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperDSize, 0#, 0#)  
    Debug.Print "  E   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperESize, 0#, 0#)  
    Debug.Print "  A4   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperA4Size, 0#, 0#)  
    Debug.Print "  A4 vert = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperA4SizeVertical, 0#, 0#)  
    Debug.Print "  A3   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperA3Size, 0#, 0#)  
    Debug.Print "  A2   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperA2Size, 0#, 0#)  
    Debug.Print "  A1   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperA1Size, 0#, 0#)  
    Debug.Print "  A0   = " & swApp.GetDocumentTemplate(swDocDRAWING, "", swDwgPaperA0Size, 0#, 0#)  
  
End Sub
```

Get and Set Loft's Pick Points Example (C#)

This example shows how to get and set the pick points of a loft feature.

```
//-----
// Preconditions:
// 1. Open a part that contains a loft.
// 2. Open the Immediate window.
// 3. Select the loft feature.
//
// Postconditions:
// 1. New loft pick points are set.
// 2. Examine the part and Immediate window to verify.
//
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace GetPickPointsLoft_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        Feature swFeat;
        LoftFeatureData swFeatData;
        MathPoint swMathPoint;
        MathPoint[] newMathPoint;
        double[] pointArray;
        Object[] pickPointData;
        Object pointVar;
        Object[] pointData;
        Object[] newChains;
        int selCount;
        int numberOfChains;
        int newCount;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            selCount = swSelMgr.GetSelectedObjectCount();
            if (selCount != 1)
            {
                System.Windows.Forms.MessageBox.Show("Select the loft feature and rerun the macro.");
                return;
            }

            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swFeatData = (LoftFeatureData)swFeat.GetDefinition();

            // Get the loft's pick points
            newCount = 0;
            pickPointData = (Object[])swFeatData.PickPoints;
            numberOfChains = pickPointData.GetUpperBound(0) + 1;
            newChains = new Object[numberOfChains];
            Debug.Print("No. of chains = " + numberOfChains);
            Debug.Print("");

            for (int chainCount = pickPointData.GetLowerBound(0); chainCount <= pickPointData.GetUpperBound(0); chainCount++)
            {
                pointData = (Object[])pickPointData[chainCount];
                int numOfPointInEachChain = pointData.GetUpperBound(0) + 1;

                MathPoint[] newPoints = new MathPoint[numOfPointInEachChain];
                Debug.Print("Chain = " + chainCount + ", Number of points = " + numOfPointInEachChain);
                for (int pointCount = pointData.GetLowerBound(0); pointCount <= pointData.GetUpperBound(0); pointCount++)
                {
                    swMathPoint = (MathPoint)pointData[pointCount];
                    pointVar = swMathPoint.ArrayData;
                    pointArray = (Double[])pointVar;
                    newPoints[pointCount] = swMathPoint;
                    Debug.Print("X = " + (double)pointArray[0] * 1000 + " Y = " + (double)pointArray[1] * 1000 + " Z = " +
                        (double)pointArray[2] * 1000);

                    newCount = newCount + 1;
                }
            }
        }
    }
}
```

```

        newChains[chainCount] = newPoints;
        Debug.Print("");
        numPoints = numOfPointInEachChain;
    }

    // Change the loft pick points, rotating them to create a twist
    MathPoint[] tmpPoints = (MathPoint[])newChains[newChains.GetUpperBound(0)];
    MathPoint objPrev = tmpPoints[numPoints - 1];
    Object[] chainArr = new Object[newChains.GetUpperBound(0) + 1];
    for (int chainCount = newChains.GetLowerBound(0); chainCount <= newChains.GetUpperBound(0); chainCount++)
    {
        MathPoint[] newPoints = (MathPoint[])newChains[chainCount];
        MathPoint tmp = newPoints[numPoints - 1];
        newPoints[numPoints - 1] = objPrev;
        objPrev = tmp;

        DispatcherWrapper[] newPointWrapArray = (DispatcherWrapper[])ObjectArrayToDispatcherWrapperArray(newPoints);
        chainArr[chainCount] = newPointWrapArray;
    }

    // Set the loft's new pick points
    Boolean status = false;
    status = swFeatData.AccessSelections(swModel, null);
    swFeatData.PickPoints = chainArr;

    status = swFeat.ModifyDefinition(swFeatData, swModel, null);
    Debug.Print("Loft's pick points were modified: " + status);
    Debug.Print("");

    // Get the loft's pick points
    newCount = 0;
    pickPointData = (Object[])swFeatData.PickPoints;
    numberOfChains = pickPointData.GetUpperBound(0) + 1;
    Debug.Print("No. of chains = " + numberOfChains);
    Debug.Print("");
    for (int chainCount = pickPointData.GetLowerBound(0); chainCount <= pickPointData.GetUpperBound(0); chainCount++)
    {
        pointData = (Object[])pickPointData[chainCount];
        long numOfPointInEachChain = pointData.GetUpperBound(0) + 1;
        Array.Resize(ref newMathPoint, (int)((numberOfChains * numOfPointInEachChain) - 1) + 1);
        Debug.Print("Chain = " + chainCount + ", Number of points = " + numOfPointInEachChain);
        for (int pointCount = pointData.GetLowerBound(0); pointCount <= pointData.GetUpperBound(0); pointCount++)
        {
            swMathPoint = (MathPoint)pointData[pointCount];
            pointVar = swMathPoint.ArrayData;
            newMathPoint[newCount] = swMathPoint;
            newMathPoint[newCount].ArrayData = pointVar;
            pointArray = (Double[])pointVar;
            Debug.Print("X = " + (double)pointArray[0] * 1000 + " Y = " + (double)pointArray[1] * 1000 + " Z = " +
                (double)pointArray[2] * 1000);
            newCount = newCount + 1;
        }
        Debug.Print("");
    }
}

public DispatcherWrapper[] ObjectArrayToDispatcherWrapperArray(object[] SwObjects)
{
    int arraySize = 0;
    arraySize = SwObjects.GetUpperBound(0);
    DispatcherWrapper[] dispwrap = new DispatcherWrapper[arraySize + 1];
    int arrayIndex = 0;
    for (arrayIndex = 0; arrayIndex <= arraySize; arrayIndex++)
    {
        dispwrap[arrayIndex] = new DispatcherWrapper(SwObjects[arrayIndex]);
    }
    return dispwrap;
}

public SldWorks swApp;
public int numPoints;
}
}

```

Get Loft's Pick Points Example (VBA)

This example shows how to get the pick points of a loft feature.

```
' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\floxpress\ball valve\handle.sldprt.
' 2. Select the feature named 1 (a loft feature).
'
' Postconditions: None
' -----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swFeatData As SldWorks.LoftFeatureData
Dim swMathPoint As SldWorks.MathPoint
Dim newMathPoint() As SldWorks.MathPoint
Dim pickPointData As Variant
Dim pointVar As Variant
Dim newTopVar As Variant
Dim pointData As Variant
Dim selCount As Long
Dim chainCount As Long
Dim pointCount As Long
Dim numberOfChains As Long
Dim numOfPointInEachChain As Long
Dim newCount As Long
Sub main()

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
selCount = swSelMgr.GetSelectedObjectCount
If selCount <> 1 Then
    MsgBox ("Select a loft feature and re-run the macro.")
End If

Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swFeatData = swFeat.GetDefinition

' Get the loft's pick points
newCount = 0
pickPointData = swFeatData.PickPoints
numberOfChains = UBound(pickPointData) + 1
Debug.Print ("No. of chains = " & (UBound(pickPointData) + 1))
Debug.Print ""
For chainCount = LBound(pickPointData) To UBound(pickPointData)
    pointData = pickPointData(chainCount)
    numOfPointInEachChain = UBound(pointData) + 1
    ReDim Preserve newMathPoint(0 To ((numberOfChains * numOfPointInEachChain) - 1))
    Debug.Print ("Chain = " & chainCount & ", Number of points = " & (UBound(pointData) + 1))
    For pointCount = LBound(pointData) To UBound(pointData)
        Set swMathPoint = pointData(pointCount)
        pointVar = swMathPoint.ArrayData
```

```
Set newMathPoint(newCount) = swMathPoint
newMathPoint(newCount).ArrayData = pointVar
Debug.Print ("X = " & pointVar(0) * 1000 & " Y = " & pointVar(1) * 1000 & " Z = " & pointVar(2) * 1000)
newCount = newCount + 1
Next pointCount
Debug.Print ""
Next chainCount
End Sub
```

Get and Set Loft's Pick Points Example (VB.NET)

This example shows how to get and set the pick points of a loft feature.

```

'-----
' Preconditions:
' 1. Open a part that contains a loft.
' 2. Open the Immediate window.
' 3. Select the loft feature.
'
' Postconditions:
' 1. New loft pick points are set.
' 2. Examine the part and the Immediate window to verify.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics
Imports System.Runtime.InteropServices

Partial Class SolidWorksMacro

    Private swModel As ModelDoc2
    Private swSelMgr As SelectionMgr
    Private swFeat As Feature
    Private swFeatData As LoftFeatureData
    Private swMathPoint As MathPoint
    Private newMathPoint As MathPoint()
    Private pointArray As Double()
    Private pickPointData As [Object]()
    Private pointVar As [Object]
    Private pointData As [Object]()
    Private newChains As [Object]()
    Private selCount As Integer
    Private numberOfChains As Integer
    Private newCount As Integer

    Public Sub Main()

        swModel = DirectCast(swApp.ActiveDoc, ModelDoc2)
        swSelMgr = DirectCast(swModel.SelectionManager, SelectionMgr)
        selCount = swSelMgr.GetSelectedObjectCount()
        If selCount <> 1 Then
            System.Windows.Forms.MessageBox.Show("Select the loft feature and rerun the macro.")
            Return
        End If

        swFeat = DirectCast(swSelMgr.GetSelectedObject6(1, -1), Feature)
        swFeatData = DirectCast(swFeat.GetDefinition(), LoftFeatureData)

        ' Get the loft's pick points
        newCount = 0
        pickPointData = DirectCast(swFeatData.PickPoints, [Object]())
        numberOfChains = pickPointData.GetUpperBound(0) + 1
        newChains = New [Object](numberOfChains - 1) {}
        Debug.Print("No. of chains = " & numberOfChains)
        Debug.Print("")

        For chainCount As Integer = pickPointData.GetLowerBound(0) To pickPointData.GetUpperBound(0)
            pointData = DirectCast(pickPointData(chainCount), [Object]())
            Dim numOfPointInEachChain As Integer = pointData.GetUpperBound(0) + 1

            Dim newPoints As MathPoint() = New MathPoint(numOfPointInEachChain - 1) {}
            Debug.Print("Chain = " & chainCount & ", Number of points = " & numOfPointInEachChain)
            For pointCount As Integer = pointData.GetLowerBound(0) To pointData.GetUpperBound(0)
                swMathPoint = DirectCast(pointData(pointCount), MathPoint)
                pointVar = swMathPoint.ArrayData
                pointArray = DirectCast(pointVar, [Double]())
                newPoints(pointCount) = swMathPoint
                Debug.Print("X = " & CDb1(pointArray(0)) * 1000 & " Y = " & CDb1(pointArray(1)) * 1000 & " Z = " &
                    CDb1(pointArray(2)) * 1000)

                newCount = newCount + 1
            Next
            newChains(chainCount) = newPoints
            Debug.Print("")
            numPoints = numOfPointInEachChain
        Next

        ' Change the loft pick points, rotating them to create a twist
        Dim tmpPoints As MathPoint() = DirectCast(newChains(newChains.GetUpperBound(0)), MathPoint())
        Dim objPrev As MathPoint = tmpPoints(numPoints - 1)
        Dim chainArr As [Object]() = New [Object](newChains.GetUpperBound(0)) {}

```

```

For chainCount As Integer = newChains.GetLowerBound(0) To newChains.GetUpperBound(0)
    Dim newPoints As MathPoint() = DirectCast(newChains(chainCount), MathPoint())
    Dim tmp As MathPoint = newPoints(numPoints - 1)
    newPoints(numPoints - 1) = objPrev
    objPrev = tmp

    Dim newPointWrapArray As DispatchWrapper() = DirectCast(ObjectArrayToDispatchWrapperArray(newPoints),
DispatchWrapper())
    chainArr(chainCount) = newPointWrapArray
Next

' Set the loft's new pick points
Dim status As [Boolean] = False
status = swFeatData.AccessSelections(swModel, Nothing)
swFeatData.PickPoints = chainArr

status = swFeat.ModifyDefinition(swFeatData, swModel, Nothing)
Debug.Print("Loft's pick points were modified: " & status)
Debug.Print("")

' Get the loft's pick points
newCount = 0
pickPointData = DirectCast(swFeatData.PickPoints, [Object]())
numberOfChains = pickPointData.GetUpperBound(0) + 1
Debug.Print("No. of chains = " & numberOfChains)
Debug.Print("")
For chainCount As Integer = pickPointData.GetLowerBound(0) To pickPointData.GetUpperBound(0)
    pointData = DirectCast(pickPointData(chainCount), [Object]())
    Dim numOfPointInEachChain As Long = pointData.GetUpperBound(0) + 1
    Array.Resize(newMathPoint, CInt((numberOfChains * numOfPointInEachChain) - 1) + 1)
    Debug.Print("Chain = " & chainCount & ", Number of points = " & numOfPointInEachChain)
    For pointCount As Integer = pointData.GetLowerBound(0) To pointData.GetUpperBound(0)
        swMathPoint = DirectCast(pointData(pointCount), MathPoint)
        pointVar = swMathPoint.ArrayData
        newMathPoint(newCount) = swMathPoint
        newMathPoint(newCount).ArrayData = pointVar
        pointArray = DirectCast(pointVar, [Double]())
        Debug.Print("X = " & CDb1(pointArray(0)) * 1000 & " Y = " & CDb1(pointArray(1)) * 1000 & " Z = " &
CDbl(pointArray(2)) * 1000)
        newCount = newCount + 1
    Next
    Debug.Print("")
Next
End Sub

Public Function ObjectArrayToDispatchWrapperArray(ByVal SwObjects As Object()) As DispatchWrapper()
    Dim arraySize As Integer = 0
    arraySize = SwObjects.GetUpperBound(0)
    Dim dispwrap As DispatchWrapper() = New DispatchWrapper(arraySize) {}
    Dim arrayIndex As Integer = 0
    For arrayIndex = 0 To arraySize
        dispwrap(arrayIndex) = New DispatchWrapper(SwObjects(arrayIndex))
    Next
    Return dispwrap
End Function

Public swApp As SldWorks
Public numPoints As Integer

End Class

```

Get Loops Example (C#)

This example shows how to get loop data for the selected face.

```
//-----
// Preconditions: Model document is open, and a face is selected.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace GetLoopEdgeCount_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Face2 swFace = default(Face2);
            Loop2 swLoop = default(Loop2);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            swLoop = (Loop2)swFace.GetFirstLoop();

            Debug.Print("LoopCount = " + swFace.GetLoopCount());

            while ((swLoop != null))
            {
                Debug.Print("  IsOuter      = " + swLoop.IsOuter());
                Debug.Print("  IsSingular  = " + swLoop.IsSingular());
                Debug.Print("  EdgeCount   = " + swLoop.GetEdgeCount());
                Debug.Print("  VertexCount = " + swLoop.GetVertexCount());
                Debug.Print("");

                swLoop = (Loop2)swLoop.GetNext();
            }
        }

        public SldWorks swApp;
    }
}
```

Get Loops Example (VBA)

This example shows how to get loop data for the selected face.

```
'-----  
' Preconditions: Model document is open, and a face is selected.  
'  
' Postconditions: Inspect the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swSelMgr              As SldWorks.SelectionMgr  
    Dim swFace               As SldWorks.Face2  
    Dim swLoop               As SldWorks.Loop2  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)  
    Set swLoop = swFace.GetFirstLoop  
  
    Debug.Print "LoopCount = " & swFace.GetLoopCount  
  
    While Not swLoop Is Nothing  
        Debug.Print "  IsOuter      = " & swLoop.IsOuter  
        Debug.Print "  IsSingular   = " & swLoop.IsSingular  
        Debug.Print "  EdgeCount    = " & swLoop.GetEdgeCount  
        Debug.Print "  VertexCount  = " & swLoop.GetVertexCount  
        Debug.Print ""  
        Set swLoop = swLoop.GetNext  
    Wend  
End Sub
```

Get Loops Example (VB.NET)

This example shows how to get loop data for the selected face.

```
'-----  
' Preconditions: Model document is open, and a face is selected.  
'-----  
' Postconditions: Inspect the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swFace As Face2  
        Dim swLoop As Loop2
```

```
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swFace = swSelMgr.GetSelectedObject6(1, -1)  
        swLoop = swFace.GetFirstLoop
```

```
        Debug.Print("LoopCount = " & swFace.GetLoopCount)
```

```
        While Not swLoop Is Nothing
```

```
            Debug.Print("  IsOuter      = " & swLoop.IsOuter)  
            Debug.Print("  IsSingular   = " & swLoop.IsSingular)  
            Debug.Print("  EdgeCount    = " & swLoop.GetEdgeCount)  
            Debug.Print("  VertexCount  = " & swLoop.GetVertexCount)  
            Debug.Print("")
```

```
            swLoop = swLoop.GetNext  
        End While
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Mask Types of Each Decal Example (VBA)

This example shows how to get a decal and its mask type.

```
'-----  
' Preconditions:  
' 1. Open a model that has at least one decal applied  
'    to it in the Default configuration.  
' 2. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swDecal As SldWorks.Decal  
Dim lNbrDecals As Long, i As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    lNbrDecals = swModelDocExt.GetDecalsCount  
    Debug.Print "Number of decals: " & lNbrDecals  
    For i = 1 To lNbrDecals  
        ' Get the decal with the corresponding decal ID in the  
        ' the Default configuration  
        Set swDecal = swModelDocExt.GetDecal(i, "Default")  
        Debug.Print "Mask type: " & swDecal.MaskType  
    Next i  
  
End Sub
```

Get Mass Properties Example (C#)

This example shows how to retrieve the mass properties of selected components in an assembly.

```
//-----
// Preconditions:
// 1. Open an assembly with one or more components.
// 2. Multi-select the components for which to get mass properties.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. Gets the mass properties for the selected components in
//    the assembly.
// 2. Inspect the Immediate window for the mass properties of
//    the selected components.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetMassProperties2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelExt = default(ModelDocExtension);

            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2 swComp = default(Component2);

            int nStatus = 0;
            double[] vMassProp = null;
            int i = 0;
            int nbrSelections = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            nbrSelections = swSelMgr.GetSelectedObjectCount2(-1);

            if (nbrSelections == 0)
            {
                Debug.Print("Please select one or more components and rerun the macro.");
                return;
            }

            nbrSelections = nbrSelections - 1;

            Debug.Print("Getting mass properties for components: ");
            for (i = 0; i <= nbrSelections; i++)
            {
                swComp = (Component2)swSelMgr.GetSelectedObject6(i + 1, -1);
                Debug.Print(" " + swComp.Name2);
            }

            vMassProp = (double[])swModelExt.GetMassProperties2(1, out nStatus, true);

            Debug.Print("Status as defined in swMassPropertiesStatus_e (0 = Mass properties calculation successful)
= " + nStatus);

            if ((vMassProp != null))
            {
                Debug.Print("Center of mass:");
                Debug.Print(" X-coordinate = " + vMassProp[0]);
                Debug.Print(" Y-coordinate = " + vMassProp[1]);
                Debug.Print(" Z-coordinate = " + vMassProp[2]);
                Debug.Print("Volume = " + vMassProp[3]);
                Debug.Print("Surface area = " + vMassProp[4]);
                Debug.Print("Mass = " + vMassProp[5]);
                Debug.Print("Density = " + vMassProp[5] / vMassProp[3]);
                Debug.Print("Moments of inertia taken at the center of mass and aligned with the output coordinate system:");
                Debug.Print(" Lxx = " + vMassProp[6]);
                Debug.Print(" Lyy = " + vMassProp[7]);
            }
        }
    }
}
```

```
Debug.Print("  Lzz = " + vMassProp[8]);  
Debug.Print("  Lxy = " + vMassProp[9]);  
Debug.Print("  Lzx = " + vMassProp[10]);  
Debug.Print("  Lyz = " + vMassProp[11]);
```

```
}
```

```
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Get Mass Properties Example (VBA)

This example shows how to retrieve the mass properties of selected components in an assembly.

```
'-----
' Preconditions:
' 1. Open an assembly with one or more components.
' 2. Multi-select components for which to get mass properties.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Gets the mass properties of the selected components
'    in the assembly.
' 2. Inspect the Immediate window for the mass properties of
'    the selected components.
'-----
```

Option Explicit

```
Sub main()
    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swModelExt           As SldWorks.ModelDocExtension
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swComp               As SldWorks.Component2
    Dim nStatus              As Long
    Dim vMassProp            As Variant
    Dim i                    As Long
    Dim nbrSelections        As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    nbrSelections = swSelMgr.GetSelectedObjectCount2(-1)

    If nbrSelections = 0 Then
        Debug.Print "Please select one or more components and rerun the macro."
        Exit Sub
    End If

    nbrSelections = nbrSelections - 1

    Debug.Print "Getting mass properties for components: "
    For i = 0 To nbrSelections
        Set swComp = swSelMgr.GetSelectedObject6(i + 1, -1)
        Debug.Print " " & swComp.Name2
    Next

    vMassProp = swModelExt.GetMassProperties2(1, nStatus, True)
    Debug.Print "Status as defined in swMassPropertiesStatus_e (0 = Mass properties calculation successful) = " & nStatus
    If Not IsEmpty(vMassProp) Then
        Debug.Print "Center of mass:"
        Debug.Print " X-coordinate = " & vMassProp(0)
        Debug.Print " Y-coordinate = " & vMassProp(1)
        Debug.Print " Z-coordinate = " & vMassProp(2)
        Debug.Print "Volume = " & vMassProp(3)
        Debug.Print "Surface area = " & vMassProp(4)
        Debug.Print "Mass = " & vMassProp(5)
        Debug.Print "Density = " & vMassProp(5) / vMassProp(3)
        Debug.Print "Moments of inertia taken at the center of mass and aligned with the output coordinate system:"
        Debug.Print " Lxx = " & vMassProp(6)
        Debug.Print " Lyy = " & vMassProp(7)
        Debug.Print " Lzz = " & vMassProp(8)
        Debug.Print " Lxy = " & vMassProp(9)
        Debug.Print " Lzx = " & vMassProp(10)
        Debug.Print " Lyz = " & vMassProp(11)

    End If
End Sub
```

Get Mass Properties Example (VB.NET)

This example shows how to retrieve the mass properties of selected components in an assembly.

```

'-----
' Preconditions:
' 1. Open an assembly with one or more components.
' 2. Multi-select the components for which to get mass properties.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Gets the mass properties of the selected components
'    in the assembly.
' 2. Inspect the Immediate window for the mass properties of
'    the selected components.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swModelExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swComp As Component2
        Dim nStatus As Integer
        Dim vMassProp As Object
        Dim i As Integer
        Dim nbrSelections As Integer

        swModel = swApp.ActiveDoc
        swModelExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        nbrSelections = swSelMgr.GetSelectedObjectCount2(-1)

        If nbrSelections = 0 Then
            Debug.Print("Please select one or more components and rerun the macro.")
            Exit Sub
        End If

        nbrSelections = nbrSelections - 1

        Debug.Print("Getting mass properties for components: ")
        For i = 0 To nbrSelections
            swComp = swSelMgr.GetSelectedObject6(i + 1, -1)
            Debug.Print(" " & swComp.Name2)
        Next

        vMassProp = swModelExt.GetMassProperties2(1, nStatus, True)

        Debug.Print("Status as defined in swMassPropertiesStatus_e (0 = Mass properties calculation successful) = " & nStatus)

        If Not IsNothing(vMassProp) Then

            Debug.Print("Center of mass:")
            Debug.Print(" X-coordinate = " & vMassProp(0))
            Debug.Print(" Y-coordinate = " & vMassProp(1))
            Debug.Print(" Z-coordinate = " & vMassProp(2))
            Debug.Print("Volume = " & vMassProp(3))
            Debug.Print("Surface area = " & vMassProp(4))
            Debug.Print("Mass = " & vMassProp(5))
            Debug.Print("Density = " & vMassProp(5) / vMassProp(3))
            Debug.Print("Moments of inertia taken at the center of mass and aligned with the output coordinate system:")
            Debug.Print(" Lxx = " & vMassProp(6))
            Debug.Print(" Lyy = " & vMassProp(7))
            Debug.Print(" Lzz = " & vMassProp(8))
            Debug.Print(" Lxy = " & vMassProp(9))
            Debug.Print(" Lzx = " & vMassProp(10))
            Debug.Print(" Lyz = " & vMassProp(11))

        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.

```

```
''' </summary>  
Public swApp As Sldworks
```

```
End Class
```

Get Mass Properties of Assembly Component Example (VBA)

An assembly component (part or sub-assembly) can contain assembly-level features, typically cuts. These features affect the mass properties, but the features are not present in the part or sub-assembly. This example shows to get the mass properties of an assembly component.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Select a component,
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the mass properties of the selected component
' 2. Examine the Immediate window.
'
' NOTE: The mass property values are returned relative to
' the component origin, not the assembly origin.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swComp As SldWorks.Component2
    Dim swCompModel As SldWorks.ModelDoc2
    Dim swCompBody As SldWorks.Body2
    Dim vMassProps As Variant
    Dim nDensity As Double
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, 0)

    Set swCompBody = swComp.GetBody
    Set swCompModel = swComp.GetModelDoc

    ' Calculate component material density
    nDensity = swCompModel.GetUserPreferenceDoubleValue(swMaterialPropertyDensity)

    ' Use this method to get component mass properties
    vMassProps = swCompBody.GetMassProperties(nDensity)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "  Comp   = " & swComp.Name2
    Debug.Print "  Config = " & swComp.ReferencedConfiguration
    Debug.Print "  Density = " & nDensity & " kg/m^3"
    Debug.Print ""
    Debug.Print "  CenterOfMass = (" & vMassProps(0) * 1000# & ", " & vMassProps(1) * 1000# & ", " & vMassProps(2) * 1000# & ") mm"
    Debug.Print "  Volume      = " & vMassProps(3) * 1000000000# & " mm^3"
    Debug.Print "  Area       = " & vMassProps(4) * 1000000# & " mm^2"
    Debug.Print "  Mass       = " & vMassProps(5) & " kg"
    Debug.Print "  MomXX      = " & vMassProps(6) & " kg*m^2"
    Debug.Print "  MomYY      = " & vMassProps(7) & " kg*m^2"
    Debug.Print "  MomZZ      = " & vMassProps(8) & " kg*m^2"
    Debug.Print "  MomXY      = " & vMassProps(9) & " kg*m^2"
    Debug.Print "  MomZX      = " & vMassProps(10) & " kg*m^2"
    Debug.Print "  MomYZ      = " & vMassProps(11) & " kg*m^2"
```

End Sub

Get Mass Properties of Inactive Document Example (VBA)

This example shows how to get the mass properties from a closed (i.e., inactive) assembly document.

```
'-----
' Preconditions:
' 1. Copy the files in public_documents\samples\tutorial\motionstudies
'    to c:\temp\motionstudies.
' 2. Open c:\temp\motionstudies\valve_cam.sldasm, click Tools > Options >
'    System Options > Performance > Update mass properties while saving
'    document > OK.
' 3. Click File > Save.
' 4. Click File > Close.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Gets the mass properties.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Const sFileName As String = "c:\temp\motionstudies\valve_cam.sldasm"
    Const sConfigName As String = "Default"
    Dim swApp As SldWorks.SldWorks
    Dim vMassProp As Variant

    Set swApp = CreateObject("SldWorks.Application")

    vMassProp = swApp.GetMassProperties2(sFileName, sConfigName, 1)
    Debug.Print "SldWorks::GetMassProperties2(" + sFileName + " [" + sConfigName + "])"
    Debug.Print "    UpdateMassPropsDuringSave = " & swApp.GetUserPreferenceToggle(swUpdateMassPropsDuringSave)
    Debug.Print ""
    If Not IsEmpty(vMassProp) Then
        Debug.Print "    CenterOfMassX = " & vMassProp(0)
        Debug.Print "    CenterOfMassY = " & vMassProp(1)
        Debug.Print "    CenterOfMassZ = " & vMassProp(2)
        Debug.Print "    Volume = " & vMassProp(3)
        Debug.Print "    Area = " & vMassProp(4)
        Debug.Print "    Mass = " & vMassProp(5)
        Debug.Print "    MomXX = " & vMassProp(6)
        Debug.Print "    MomYY = " & vMassProp(7)
        Debug.Print "    MomZZ = " & vMassProp(8)
        Debug.Print "    MomXY = " & vMassProp(9)
        Debug.Print "    MomZX = " & vMassProp(10)
        Debug.Print "    MomYZ = " & vMassProp(11)
    End If
End Sub
```

Get Mass Properties of Multibody Assembly Component Example (C#)

This example shows how to get the mass properties of a multibody assembly component in which an assembly cut-extrude feature is created.

NOTES:

- An assembly component, i.e., a part or subassembly, can contain one or more assembly-level features. Some types of assembly features, e.g., cut extrude, can affect the mass properties. Assembly features are not present in the part or subassembly.
- Mass property values returned are relative to the component origin, not the assembly origin.

```
//-----
// Preconditions:
// 1. Verify that the specified multibody part document
//    and assembly document template exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified multibody part document.
// 2. Creates an assembly using the specified multibody
//    part document.
// 3. Creates an assembly cut-extrude feature.
// 4. Selects the multibody component.
// 5. Gets the mass property values of the multibody
//    component.
// 6. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            ModelDocExtension swDocExt = default(ModelDocExtension);
            MassProperty swMass = default(MassProperty);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2 swComp = default(Component2);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            object vBodyInfo = null;
            double[] vCoM = null;
            double[] vMoI = null;
            double[] vPrinAoIx = null;
            double[] vPrinAoIy = null;
            double[] vPrinAoIz = null;
            double[] vPrinMoI = null;
            bool bRet = false;
            int errors = 0;
            int warnings = 0;

            //Open multibody part document and create an assembly
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\multi_int
er.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2017\\templates\\Assembly.asmdot", 0, 0, 0);
            swAssembly = (AssemblyDoc)swModel;
            swComp = (Component2)swAssembly.AddComponent5("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\
\\multi_inter.sldprt", (int)swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", false, "", -9.26777909171506E-
05, 0, 4.8904806817518E-05);

            //Create an assembly cut-extrude feature
            swDocExt = (ModelDocExtension)swModel.Extension;
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swFeatMgr = (FeatureManager)swModel.FeatureManager;
            bRet = swDocExt.SelectByID2("", "FACE", -0.0195381300573558, 0.04499999999998454, -0.00303401890568011, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.0, 0.0, 0.0, 0.002956, -0.004701, 0.0);
            swModel.ClearSelection2(true);
            bRet = swDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swFeat = (Feature)swFeatMgr.FeatureCut3(true, false, false, 0, 0, 0.5, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, true, true,
            true, true, false, 0, 0, false);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelMgr.EnableContourSelection = false;

            //Select multibody component
            bRet = swDocExt.SelectByID2("multi_inter-1@Assem1", "COMPONENT", 0, 0, 0, false, 0, null, 0);

            swMass = (MassProperty)swDocExt.CreateMassProperty();
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent4(1, -1);
            object[] vBodies = new object[1];
            vBodies = (object[])swComp.GetBodies3((int)swBodyType_e.swSolidBody, out vBodyInfo);
            Dispatcher[] dispArray = ObjectArrayToDispatcherArray(vBodies);
            bRet = swMass.AddBodies(dispArray);
        }
    }
}
```

```

//Get mass properties of selected component's bodies
vCoM = (double[])swMass.CenterOfMass;
vMoI = (double[])swMass.GetMomentOfInertia((int)swMassPropertyMoment_e.swMassPropertyMomentAboutCenterOfMass);
vPrinAoIx = (double[])swMass.get_PrincipleAxesOfInertia(0);
vPrinAoIy = (double[])swMass.get_PrincipleAxesOfInertia(1);
vPrinAoIz = (double[])swMass.get_PrincipleAxesOfInertia(2);
vPrinMoI = (double[])swMass.PrincipleMomentsOfInertia;
Debug.Print("Component = " + swComp.Name2);
Debug.Print("Configuration = " + swComp.ReferencedConfiguration);
Debug.Print("Density = " + swMass.Density + " kg/m^3");
Debug.Print("");
Debug.Print("Center of mass = (" + vCoM[0] * 1000.0 + ", " + vCoM[1] * 1000.0 + ", " + vCoM[2] * 1000.0 + ") mm");
Debug.Print("Volume = " + swMass.Volume * 1000000000.0 + " mm^3");
Debug.Print("Area = " + swMass.SurfaceArea * 1000000.0 + " mm^2");
Debug.Print("Mass = " + swMass.Mass + " kg");
Debug.Print("Principle axes of inertia ");
Debug.Print(" Ix = (" + vPrinAoIx[0] + ", " + vPrinAoIx[1] + ", " + vPrinAoIx[2] + ")");
Debug.Print(" Iy = (" + vPrinAoIy[0] + ", " + vPrinAoIy[1] + ", " + vPrinAoIy[2] + ")");
Debug.Print(" Iz = (" + vPrinAoIz[0] + ", " + vPrinAoIz[1] + ", " + vPrinAoIz[2] + ")");
Debug.Print("Principle moments of inertia");
Debug.Print(" Px = " + vPrinMoI[0] + " kg*m^2");
Debug.Print(" Py = " + vPrinMoI[1] + " kg*m^2");
Debug.Print(" Pz = " + vPrinMoI[2] + " kg*m^2");
Debug.Print("Products of inertia");
Debug.Print(" Lxx = " + vMoI[0] + " kg*m^2");
Debug.Print(" Lxy = " + vMoI[1] + " kg*m^2");
Debug.Print(" Lxz = " + vMoI[2] + " kg*m^2");
Debug.Print(" Lyx = " + vMoI[3] + " kg*m^2");
Debug.Print(" Lyy = " + vMoI[4] + " kg*m^2");
Debug.Print(" Lyz = " + vMoI[5] + " kg*m^2");
Debug.Print(" Lzx = " + vMoI[6] + " kg*m^2");
Debug.Print(" Lzy = " + vMoI[7] + " kg*m^2");
Debug.Print(" Lzz = " + vMoI[8] + " kg*m^2");
}
public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
{
    int ArraySize = 0;
    ArraySize = Objects.GetUpperBound(0);
    Dispatcher[] d = new Dispatcher[ArraySize + 1];
    int ArrayIndex = 0;
    for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
    {
        d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
    }
    return d;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Mass Properties of Multibody Assembly Component Example (VBA)

This example shows how to get the mass properties of a multibody assembly component in which an assembly cut-extrude feature is created.

NOTES:

- An assembly component, i.e., a part or subassembly, can contain one or more assembly-level features. Some types of assembly features, e.g., cut extrude, can affect the mass properties. Assembly features are not present in the part or subassembly.
- Mass property values returned are relative to the component origin, not the assembly origin.

```
'-----
' Preconditions:
' 1. Verify that the specified multibody part document
'   and assembly document template exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified multibody part document.
' 2. Creates an assembly using the specified multibody
'   part document.
' 3. Creates an assembly cut-extrude feature.
' 4. Selects the multibody component.
' 5. Gets the mass property values of the multibody
'   component.
' 6. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssembly As SldWorks.AssemblyDoc
    Dim swDocExt As SldWorks.ModelDocExtension
    Dim swMass As SldWorks.MassProperty
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swComp As SldWorks.Component2
    Dim swSketchMgr As SldWorks.SketchManager
    Dim swSketchSegment As SldWorks.SketchSegment
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swFeat As SldWorks.Feature
    Dim vBodyArr As Variant
    Dim vBodyInfo As Variant
    Dim vCoM As Variant
    Dim vMoI As Variant
    Dim vPrinAoIx As Variant
    Dim vPrinAoIy As Variant
    Dim vPrinAoIz As Variant
    Dim vPrinMoI As Variant
    Dim bRet As Boolean
    Dim errors As Long
    Dim warnings As Long

    Set swApp = Application.SldWorks

    'Open multibody part document and create an assembly
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt",
swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2017\templates\Assembly.asmdot", 0, 0, 0)
    Set swAssembly = swModel
    Set swComp = swAssembly.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt",
swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", -9.26777909171506E-05, 0, 4.8904806817518E-05)

    'Create an assembly cut-extrude feature
    Set swDocExt = swModel.Extension
    Set swSketchMgr = swModel.SketchManager
    Set swFeatMgr = swModel.FeatureManager
    bRet = swDocExt.SelectByID("", "FACE", -1.95381300573558E-02, 4.49999999998454E-02, -3.03401890568011E-03, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.002956, -0.004701, 0#)
    swModel.ClearSelection2 True
    bRet = swDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeat = swFeatMgr.FeatureCut3(True, False, False, 0, 0, 0.5, 0.00254, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, False, True, True, True, True, False, 0, 0, False)
    Set swSelMgr = swModel.SelectionManager
    swSelMgr.EnableContourSelection = False

    'Select multibody component
    bRet = swDocExt.SelectByID2("multi_inter-1@Assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)

    Set swMass = swDocExt.CreateMassProperty
    Set swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)

    vBodyArr = swComp.GetBodies3(swSolidBody, vBodyInfo)
    bRet = swMass.AddBodies((vBodyArr))

    'Get mass properties of selected component's bodies
    vCoM = swMass.CenterOfMass
    vMoI = swMass.GetMomentOfInertia(swMassPropertyMomentAboutCenterOfMass)
```

```

vPrinAoIx = swMass.PrincipleAxesOfInertia(0)
vPrinAoIy = swMass.PrincipleAxesOfInertia(1)
vPrinAoIz = swMass.PrincipleAxesOfInertia(2)
vPrinMoI = swMass.PrincipleMomentsOfInertia
Debug.Print "Component = " & swComp.Name2
Debug.Print "Configuration = " & swComp.ReferencedConfiguration
Debug.Print "Density = " & swMass.Density & " kg/m^3"
Debug.Print ""
Debug.Print "Center of mass = (" & vCoM(0) * 1000# & ", " & vCoM(1) * 1000# & ", " & vCoM(2) * 1000# & ") mm"
Debug.Print "Volume = " & swMass.Volume * 1000000000# & " mm^3"
Debug.Print "Area = " & swMass.SurfaceArea * 1000000# & " mm^2"
Debug.Print "Mass = " & swMass.Mass & " kg"
Debug.Print "Principle axes of inertia "
Debug.Print "  Ix = (" & vPrinAoIx(0) & ", " & vPrinAoIx(1) & ", " & vPrinAoIx(2) & ")"
Debug.Print "  Iy = (" & vPrinAoIy(0) & ", " & vPrinAoIy(1) & ", " & vPrinAoIy(2) & ")"
Debug.Print "  Iz = (" & vPrinAoIz(0) & ", " & vPrinAoIz(1) & ", " & vPrinAoIz(2) & ")"
Debug.Print "Principle moments of inertia"
Debug.Print "  Px = " & vPrinMoI(0) & " kg*m^2"
Debug.Print "  Py = " & vPrinMoI(1) & " kg*m^2"
Debug.Print "  Pz = " & vPrinMoI(2) & " kg*m^2"
Debug.Print "Products of inerita"
Debug.Print "  Lxx = " & vMoI(0) & " kg*m^2"
Debug.Print "  Lxy = " & vMoI(1) & " kg*m^2"
Debug.Print "  Lxz = " & vMoI(2) & " kg*m^2"
Debug.Print "  Lyx = " & vMoI(3) & " kg*m^2"
Debug.Print "  Lyy = " & vMoI(4) & " kg*m^2"
Debug.Print "  Lyz = " & vMoI(5) & " kg*m^2"
Debug.Print "  Lzx = " & vMoI(6) & " kg*m^2"
Debug.Print "  Lzy = " & vMoI(7) & " kg*m^2"
Debug.Print "  Lzz = " & vMoI(8) & " kg*m^2"

```

End Sub

Get Mass Properties of Multibody Assembly Component Example (VB.NET)

This example shows how to get the mass properties of a multibody assembly component in which an assembly cut-extrude feature is created.

NOTES:

- An assembly component, i.e., a part or subassembly, can contain one or more assembly-level features. Some types of assembly features, e.g., cut extrude, can affect the mass properties. Assembly features are not present in the part or subassembly.
- Mass property values returned are relative to the component origin, not the assembly origin.

```

'-----
' Preconditions:
' 1. Verify that the specified multibody part document
'    and assembly document template exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified multibody part document.
' 2. Creates an assembly using the specified multibody
'    part document.
' 3. Creates an assembly cut-extrude feature.
' 4. Selects the multibody component.
' 5. Gets the mass property values of the multibody
'    component.
' 6. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDocExt As ModelDocExtension
        Dim swAssembly As AssemblyDoc
        Dim swMass As MassProperty
        Dim swSelMgr As SelectionMgr
        Dim swComp As Component2
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatMgr As FeatureManager
        Dim swFeat As Feature
        Dim vBodyInfo As Object = Nothing
        Dim vCoM() As Double
        Dim vMoI() As Double
        Dim vPrinAoIx() As Double
        Dim vPrinAoIy() As Double
        Dim vPrinAoIz() As Double
        Dim vPrinMoI() As Double
        Dim bRet As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        'Open multibody part document and create an assembly
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt", swDocumentTy
pes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2017\templates\Assembly.asmdot", 0, 0, 0)
        swAssembly = swModel
        swComp = swAssembly.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt", swA
ddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", -0.0000926777909171506, 0, 0.000048904806817518)

        'Create an assembly cut-extrude feature
        swDocExt = swModel.Extension
        swSketchMgr = swModel.SketchManager
        swFeatMgr = swModel.FeatureManager
        bRet = swDocExt.SelectByID2("", "FACE", -0.0195381300573558, 0.0449999999998454, -0.00303401890568011, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchMgr.CreateCircle(0.0#, 0.0#, 0.0#, 0.002956, -0.004701, 0.0#)
        swModel.ClearSelection2(True)
        bRet = swDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        swFeat = swFeatMgr.FeatureCut3(True, False, False, 0, 0, 0.5, 0.00254, False, False, False, False, 0.0174532925199433, 0.0174532925199433,
False, False, False, False, False, True, True, False, 0, 0, False)
        swSelMgr = swModel.SelectionManager
        swSelMgr.EnableContourSelection = False

        'Select multibody component
        bRet = swDocExt.SelectByID2("multi_inter-1@Assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)

        swMass = swDocExt.CreateMassProperty
        swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)
        Dim vBodies(0) As Object
        vBodies = swComp.GetBodies3(swBodyType_e.swSolidBody, vBodyInfo)
        Dim dispArray() As DispatcherWrapper
        dispArray = ObjectArrayToDispatcherWrapperArray(vBodies)
        bRet = swMass.AddBodies(dispArray)

        'Get mass properties of selected component's bodies
        vCoM = swMass.CenterOfMass
        vMoI = swMass.GetMomentOfInertia(swMassPropertyMoment_e.swMassPropertyMomentAboutCenterOfMass)
        vPrinAoIx = swMass.PrincipleAxesOfInertia(0)
        vPrinAoIy = swMass.PrincipleAxesOfInertia(1)
        vPrinAoIz = swMass.PrincipleAxesOfInertia(2)
        vPrinMoI = swMass.PrincipleMomentsOfInertia
        Debug.Print("Component = " & swComp.Name2)
        Debug.Print("Configuration = " & swComp.ReferencedConfiguration)
        Debug.Print("Density = " & swMass.Density & " kg/m^3")
        Debug.Print("")
        Debug.Print("Center of mass = (" & vCoM(0) * 1000.0# & ", " & vCoM(1) * 1000.0# & ", " & vCoM(2) * 1000.0# & ") mm")
    End Sub
End Class

```

```

Debug.Print("Volume = " & swMass.Volume * 1000000000.0# & " mm^3")
Debug.Print("Area = " & swMass.SurfaceArea * 1000000.0# & " mm^2")
Debug.Print("Mass = " & swMass.Mass & " kg")
Debug.Print("Principle axes of inertia ")
Debug.Print(" Ix = (" & vPrinAoIx(0) & ", " & vPrinAoIx(1) & ", " & vPrinAoIx(2) & ")")
Debug.Print(" Iy = (" & vPrinAoIy(0) & ", " & vPrinAoIy(1) & ", " & vPrinAoIy(2) & ")")
Debug.Print(" Iz = (" & vPrinAoIz(0) & ", " & vPrinAoIz(1) & ", " & vPrinAoIz(2) & ")")
Debug.Print("Principle moments of inertia")
Debug.Print(" Px = " & vPrinMoi(0) & " kg*m^2")
Debug.Print(" Py = " & vPrinMoi(1) & " kg*m^2")
Debug.Print(" Pz = " & vPrinMoi(2) & " kg*m^2")
Debug.Print("Products of inertia")
Debug.Print(" Lxx = " & vMoi(0) & " kg*m^2")
Debug.Print(" Lxy = " & vMoi(1) & " kg*m^2")
Debug.Print(" Lxz = " & vMoi(2) & " kg*m^2")
Debug.Print(" Lyx = " & vMoi(3) & " kg*m^2")
Debug.Print(" Lyy = " & vMoi(4) & " kg*m^2")
Debug.Print(" Lyz = " & vMoi(5) & " kg*m^2")
Debug.Print(" Lzx = " & vMoi(6) & " kg*m^2")
Debug.Print(" Lzy = " & vMoi(7) & " kg*m^2")
Debug.Print(" Lzz = " & vMoi(8) & " kg*m^2")

```

End Sub

```

Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
    Dim ArraySize As Integer
    ArraySize = Objects.GetUpperBound(0)
    Dim d(ArraySize) As DispatchWrapper
    Dim ArrayIndex As Integer
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
    Next
    Return d
End Function

```

Public swApp As SldWorks

End Class

Get Mass Properties of Visible and Hidden Components Example (C#)

This example shows how to get the mass properties for:

- visible and hidden components in an assembly.
- only the visible components in an assembly.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\routing-pipes\ball valve with flanges.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the mass properties of all components.
// 2. Hides the slip on weld flange<1> component.
// 3. Gets the mass properties of the visible components only.
// 4. Examine the Immediate window.
//
// NOTE: Because this assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace IncludeMassPropertiesOfHiddenBodiesModelDocExtCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            int massStatus = 0;
            double[] massProperties;
            bool boolstatus;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;

            Debug.Print("-----");
            Debug.Print("Mass properties of visible and hidden components:");
            massProperties = (double[])swModelDocExt.GetMassProperties(1, ref massStatus);
            if ((massProperties != null))
            {
                Debug.Print(" CenterOfMassX = " + massProperties[0]);
                Debug.Print(" CenterOfMassY = " + massProperties[1]);
                Debug.Print(" CenterOfMassZ = " + massProperties[2]);
                Debug.Print(" Volume = " + massProperties[3]);
                Debug.Print(" Area = " + massProperties[4]);
                Debug.Print(" Mass = " + massProperties[5]);
                Debug.Print(" MomXX = " + massProperties[6]);
                Debug.Print(" MomYY = " + massProperties[7]);
                Debug.Print(" MomZZ = " + massProperties[8]);
                Debug.Print(" MomXY = " + massProperties[9]);
                Debug.Print(" MomZX = " + massProperties[10]);
                Debug.Print(" MomYZ = " + massProperties[11]);
            }

            Debug.Print("-----");

            // Now hide another component
            boolstatus = swModelDocExt.SelectByID2("slip on weld flange-
1@ball valve with flanges", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swModel.HideComponent2();

            Debug.Print("Mass properties of visible components only:");
            swModelDocExt.IncludeMassPropertiesOfHiddenBodies = false;
            massProperties = (double[])swModelDocExt.GetMassProperties(1, ref massStatus);
            if ((massProperties != null))
            {
                Debug.Print(" CenterOfMassX = " + massProperties[0]);
                Debug.Print(" CenterOfMassY = " + massProperties[1]);
                Debug.Print(" CenterOfMassZ = " + massProperties[2]);
                Debug.Print(" Volume = " + massProperties[3]);
                Debug.Print(" Area = " + massProperties[4]);
                Debug.Print(" Mass = " + massProperties[5]);
                Debug.Print(" MomXX = " + massProperties[6]);
                Debug.Print(" MomYY = " + massProperties[7]);
                Debug.Print(" MomZZ = " + massProperties[8]);
                Debug.Print(" MomXY = " + massProperties[9]);
                Debug.Print(" MomZX = " + massProperties[10]);
                Debug.Print(" MomYZ = " + massProperties[11]);
            }
            Debug.Print("-----");
        }
    }
}
```

```
}  
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}  
}
```

Get Mass Properties of Visible and Hidden Components Example (VBA)

This example shows how to get the mass properties for:

- visible and hidden components in an assembly.
- only the visible components in an assembly.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\routing-pipes\ball valve with flanges.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the mass properties of all components.
' 2. Hides the slip on weld flange<1> component.
' 3. Gets the mass properties of the visible components only.
' 4. Examine the Immediate window.
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim massStatus As Long
Dim vMassProp As Variant
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    Debug.Print "-----"

    Debug.Print "Mass properties of all visible and hidden components:"
    vMassProp = swModelDocExt.GetMassProperties(1, massStatus)
    If Not IsEmpty(vMassProp) Then
        Debug.Print " CenterOfMassX = " & vMassProp(0)
        Debug.Print " CenterOfMassY = " & vMassProp(1)
        Debug.Print " CenterOfMassZ = " & vMassProp(2)
        Debug.Print " Volume = " & vMassProp(3)
        Debug.Print " Area = " & vMassProp(4)
        Debug.Print " Mass = " & vMassProp(5)
        Debug.Print " MomXX = " & vMassProp(6)
        Debug.Print " MomYY = " & vMassProp(7)
        Debug.Print " MomZZ = " & vMassProp(8)
        Debug.Print " MomXY = " & vMassProp(9)
        Debug.Print " MomZX = " & vMassProp(10)
        Debug.Print " MomYZ = " & vMassProp(11)
    End If

    Debug.Print "-----"

    ' Now hide a component
    boolstatus = swModelDocExt.SelectByID2("slip on weld flange-1@ball valve with flanges", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swModel.HideComponent2

    Debug.Print "Mass properties of visible components only:"
    swModelDocExt.IncludeMassPropertiesOfHiddenBodies = False
    vMassProp = swModelDocExt.GetMassProperties(1, massStatus)
    If Not IsEmpty(vMassProp) Then
        Debug.Print " CenterOfMassX = " & vMassProp(0)
        Debug.Print " CenterOfMassY = " & vMassProp(1)
        Debug.Print " CenterOfMassZ = " & vMassProp(2)
        Debug.Print " Volume = " & vMassProp(3)
        Debug.Print " Area = " & vMassProp(4)
        Debug.Print " Mass = " & vMassProp(5)
        Debug.Print " MomXX = " & vMassProp(6)
        Debug.Print " MomYY = " & vMassProp(7)
        Debug.Print " MomZZ = " & vMassProp(8)
        Debug.Print " MomXY = " & vMassProp(9)
        Debug.Print " MomZX = " & vMassProp(10)
        Debug.Print " MomYZ = " & vMassProp(11)
    End If

    Debug.Print "-----"

End Sub

```

Get Mass Properties of Visible and Hidden Components Example (VB.NET)

This example shows how to get the mass properties for:

- visible and hidden components in an assembly.
- only the visible components in an assembly.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\routing-pipes\ball valve with flanges.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the mass properties of all components.
' 2. Hides the slip on weld flange<1> component.
' 3. Gets the mass properties of the visible components only.
' 4. Examine the Immediate window.
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim massStatus As Long
        Dim vMassProp As Object
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        Debug.Print("-----")
        Debug.Print("Mass properties of visible and hidden components:")
        vMassProp = swModelDocExt.GetMassProperties(1, massStatus)
        If Not vMassProp Is Nothing Then
            Debug.Print("  CenterOfMassX = " & vMassProp(0))
            Debug.Print("  CenterOfMassY = " & vMassProp(1))
            Debug.Print("  CenterOfMassZ = " & vMassProp(2))
            Debug.Print("  Volume = " & vMassProp(3))
            Debug.Print("  Area = " & vMassProp(4))
            Debug.Print("  Mass = " & vMassProp(5))
            Debug.Print("  MomXX = " & vMassProp(6))
            Debug.Print("  MomYY = " & vMassProp(7))
            Debug.Print("  MomZZ = " & vMassProp(8))
            Debug.Print("  MomXY = " & vMassProp(9))
            Debug.Print("  MomZX = " & vMassProp(10))
            Debug.Print("  MomYZ = " & vMassProp(11))
        End If
        Debug.Print("-----")

        ' Now hide another component
        boolstatus = swModelDocExt.SelectByID2("slip on weld flange-1@ball valve with flanges", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swModel.HideComponent2()

        Debug.Print("Mass properties of visible components only:")
        swModelDocExt.IncludeMassPropertiesOfHiddenBodies = False
        vMassProp = swModelDocExt.GetMassProperties(1, massStatus)
        If Not vMassProp Is Nothing Then
            Debug.Print("  CenterOfMassX = " & vMassProp(0))
            Debug.Print("  CenterOfMassY = " & vMassProp(1))
            Debug.Print("  CenterOfMassZ = " & vMassProp(2))
            Debug.Print("  Volume = " & vMassProp(3))
            Debug.Print("  Area = " & vMassProp(4))
            Debug.Print("  Mass = " & vMassProp(5))
            Debug.Print("  MomXX = " & vMassProp(6))
            Debug.Print("  MomYY = " & vMassProp(7))
            Debug.Print("  MomZZ = " & vMassProp(8))
            Debug.Print("  MomXY = " & vMassProp(9))
            Debug.Print("  MomZX = " & vMassProp(10))
            Debug.Print("  MomYZ = " & vMassProp(11))
        End If
        Debug.Print("-----")
    End Sub

''' <summary>

''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get and Override Mass Properties Example (C#)

This example shows how to get and override some mass properties of a part.

```
// -----
// Preconditions:
// 1. Ensure the specified document exists.
// 2. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window.
// -----

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace MassProperty2_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDocExtension Extn;
        private MassProperty2 MyMassProp;
        private MassPropertyOverrideOptions OvProp;
        private ModelDoc2 swModelDoc;
        private double[] pmoi;
        private double[] vValue;
        private double[] value = new double[3];
        private int errors;
        private int warnings;
        private double val;

        public void Main()
        {
            swModelDoc = swApp.OpenDoc6(@"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2019\samples\tutorial\advdrawings\98Food processor.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            Extn = swModelDoc.Extension;

            // Create mass properties and override options
            MyMassProp = (MassProperty2)Extn.CreateMassProperty2();
            OvProp = (MassPropertyOverrideOptions)MyMassProp.GetOverrideOptions();
            OvProp.OverrideMass = true;
            OvProp.SetOverrideMassValue(100);
            double[] comArr = new double[9];
            comArr[0] = 0.1677;
            comArr[1] = 0;
            comArr[2] = 0;
        }
    }
}
```

```

comArr[3] = 0;
comArr[4] = 0.21358;
comArr[5] = 0;
comArr[6] = 0;
comArr[7] = 0;
comArr[8] = 0.34772;
OvProp.OverrideMomentsOfInertia = true;
OvProp.SetOverrideMomentsOfInertiaValue(0, comArr, "");

// Use document property units (MKS)
MyMassProp.UseSystemUnits = false;

Debug.Print("Mass properties before override");
Debug.Print("");

val = MyMassProp.Mass;
Debug.Print("Mass: " + val);
val = MyMassProp.Volume;
Debug.Print("Volume: " + val);
val = MyMassProp.Density;
Debug.Print("Density: " + val);
val = MyMassProp.SurfaceArea;
Debug.Print("Surface area: " + val);
pmoi = (double[])MyMassProp.PrincipalMomentsOfInertia;
Debug.Print("Principal moments of inertia: Px: " + pmoi[0] + ", Py: " + pmoi[1] + ", and Pz: " + pmoi[2]);
vValue = (double[])MyMassProp.GetMomentOfInertia(0);
Debug.Print("Moments of inertia: Lxx: " + vValue[0] + ", Lxy: " + vValue[1] + ", Lxz: " + vValue[2] + ", Lyx: " + vValue[3] + ",
Lyy: " + vValue[4] + ", Lyz: " + vValue[5] + ", Lzx: " + vValue[6] + ", Lzy: " + vValue[7] + ", Lzz: " +
vValue[8]);

MyMassProp.SetOverrideOptions(OvProp, (int)swInConfigurationOpts_e.swThisConfiguration, null);
MyMassProp.Recalculate();

Debug.Print("");
Debug.Print("Mass properties after override");
Debug.Print("");

val = MyMassProp.Mass;
Debug.Print("Mass: " + val);
val = MyMassProp.Volume;
Debug.Print("Volume: " + val);
val = MyMassProp.Density;
Debug.Print("Density: " + val);
val = MyMassProp.SurfaceArea;
Debug.Print("Surface area: " + val);
pmoi = (double[])MyMassProp.PrincipalMomentsOfInertia;
Debug.Print("Principal moments of inertia: Px: " + pmoi[0] + ", Py: " + pmoi[1] + ", and Pz: " + pmoi[2]);
vValue = (double[])MyMassProp.GetMomentOfInertia(0);
Debug.Print("Moments of inertia: Lxx: " + vValue[0] + ", Lxy: " + vValue[1] + ", Lxz: " + vValue[2] + ", Lyx: " + vValue[3] + ",
Lyy: " + vValue[4] + ", Lyz: " + vValue[5] + ", Lzx: " + vValue[6] + ", Lzy: " + vValue[7] + ", Lzz: " +
vValue[8]);
}
/// <summary>
///     ''' The SldWorks swApp variable is pre-assigned for you.
///     ''' </summary>
public SldWorks swApp;
}
}

```

Get and Override Mass Properties Example (VB.NET)

This example shows how to get and override some mass properties of a part.

```
' -----
' Preconditions:
' 1. Ensure the specified document exists.
' 2. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window.
' -----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
    Dim Extn As ModelDocExtension
    Dim MyMassProp As MassProperty2
    Dim OvProp As MassPropertyOverrideOptions
    Dim swModelDoc As ModelDoc2
    Dim pmoi As Object
    Dim vValue As Object
    Dim value(2) As Double
    Dim errors As Integer
    Dim warnings As Integer
    Dim val As Double

    Sub main()

        swModelDoc = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\advdrawings\98food
processor.sldasm", swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        Extn = swModelDoc.Extension

        ' Create mass properties and override options
        MyMassProp = Extn.CreateMassProperty2
        OvProp = MyMassProp.GetOverrideOptions
        OvProp.OverrideMass = True
        OvProp.SetOverrideMassValue(100)
        Dim comArr(8) As Double
        comArr(0) = 0.1677
        comArr(1) = 0
        comArr(2) = 0
        comArr(3) = 0
        comArr(4) = 0.21358
        comArr(5) = 0
        comArr(6) = 0
        comArr(7) = 0
        comArr(8) = 0.34772
        OvProp.OverrideMomentsOfInertia = True
        OvProp.SetOverrideMomentsOfInertiaValue(0, comArr, "")

        ' Use document property units (MKS)
        MyMassProp.UseSystemUnits = False
```

```

Debug.Print("Mass properties before override")
Debug.Print("")

val = MyMassProp.Mass
Debug.Print("Mass: " & val)
val = MyMassProp.Volume
Debug.Print("Volume: " & val)
val = MyMassProp.Density
Debug.Print("Density: " & val)
val = MyMassProp.SurfaceArea
Debug.Print("Surface area: " & val)
pmoi = MyMassProp.PrincipalMomentsOfInertia
Debug.Print("Principal moments of inertia: Px: " & pmoi(0) & ", Py: " & pmoi(1) & ", and Pz: " & pmoi(2))
vValue = MyMassProp.GetMomentOfInertia(0)
Debug.Print("Moments of inertia: Lxx: " & vValue(0) & ", Lxy: " & vValue(1) & ", Lxz: " & vValue(2) & ", Lyx: " &
vValue(3) & ", Lyy: " & vValue(4) & ", Lyz: " & vValue(5) & ", Lzx: " & vValue(6) & ", Lzy: " & vValue(7) & ", Lzz: " &
vValue(8))

MyMassProp.SetOverrideOptions(OvProp, swInConfigurationOpts_e.swThisConfiguration, Nothing)
MyMassProp.Recalculate

Debug.Print("")
Debug.Print("Mass properties after override")
Debug.Print("")

val = MyMassProp.Mass
Debug.Print("Mass: " & val)
val = MyMassProp.Volume
Debug.Print("Volume: " & val)
val = MyMassProp.Density
Debug.Print("Density: " & val)
val = MyMassProp.SurfaceArea
Debug.Print("Surface area: " & val)
pmoi = MyMassProp.PrincipalMomentsOfInertia
Debug.Print("Principal moments of inertia: Px: " & pmoi(0) & ", Py: " & pmoi(1) & ", and Pz: " & pmoi(2))
vValue = MyMassProp.GetMomentOfInertia(0)
Debug.Print("Moments of inertia: Lxx: " & vValue(0) & ", Lxy: " & vValue(1) & ", Lxz: " & vValue(2) & ", Lyx: " &
vValue(3) & ", Lyy: " & vValue(4) & ", Lyz: " & vValue(5) & ", Lzx: " & vValue(6) & ", Lzy: " & vValue(7) & ", Lzz: " &
vValue(8))

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Get and Override Mass Properties Example (VBA)

This example shows how to get and override some mass properties of a part.

```
'-----  
' Preconditions:  
' 1. Ensure the specified document exists.  
' 2. Open the Immediate window.  
'  
' Postconditions: Inspect the Immediate window.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Extn As SldWorks.ModelDocExtension  
Dim MyMassProp As SldWorks.MassProperty2  
Dim OvProp As SldWorks.MassPropertyOverrideOptions  
Dim swModelDoc As SldWorks.ModelDoc2  
Dim pmoi As Variant  
Dim vValue As Variant  
Dim value(2) As Double  
Dim errors As Long  
Dim warnings As Long  
Dim val As Double  
Dim params As Variant  
  
Option Explicit  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModelDoc = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\advdrawings\98food processor.sldasm", swDocASSEMBLY, swOpenDocOptions_Silent,  
    "", errors, warnings)  
    Set Extn = swModelDoc.Extension  
  
    ' Create mass properties and override options  
    Set MyMassProp = Extn.CreateMassProperty2  
    Set OvProp = MyMassProp.GetOverrideOptions  
    OvProp.OverrideMass = True  
    OvProp.SetOverrideMassValue (100)  
    Dim comArr(8) As Double  
    comArr(0) = 0.1677  
    comArr(1) = 0  
    comArr(2) = 0  
    comArr(3) = 0  
    comArr(4) = 0.21358  
    comArr(5) = 0  
    comArr(6) = 0  
    comArr(7) = 0  
    comArr(8) = 0.34772  
    OvProp.OverrideMomentsOfInertia = True  
    OvProp.SetOverrideMomentsOfInertiaValue 0, comArr, ""  
  
    ' Use document property units (MKS)  
    MyMassProp.UseSystemUnits = False  
  
    Debug.Print "Mass properties before override"  
    Debug.Print ""  
  
    val = MyMassProp.Mass  
    Debug.Print "Mass: " & val  
    val = MyMassProp.Volume  
    Debug.Print "Volume: " & val  
    val = MyMassProp.Density  
    Debug.Print "Density: " & val  
    val = MyMassProp.SurfaceArea  
    Debug.Print "Surface area: " & val  
    pmoi = MyMassProp.PrincipalMomentsOfInertia
```

```
Debug.Print "Principal moments of inertia: Px: " & pmoi(0) & ", Py: " & pmoi(1) & ", and Pz: " & pmoi(2)

vValue = MyMassProp.GetMomentOfInertia(0)

Debug.Print "Moments of inertia: Lxx: " & vValue(0) & ", Lxy: " & vValue(1) & ", Lxz: " & vValue(2) & ", Lyx: " & vValue(3) & ", Lyy: " & vValue(4) & ", Lyz: " & vValue(5) & ", Lzx: " & vValue(6) & ", Lzy: " & vValue(7)
& ", Lzz: " & vValue(8)

MyMassProp.SetOverrideOptions OvProp, swThisConfiguration, Empty
MyMassProp.Recalculate

Debug.Print ""
Debug.Print "Mass properties after override"
Debug.Print ""

val = MyMassProp.Mass
Debug.Print "Mass: " & val
val = MyMassProp.Volume
Debug.Print "Volume: " & val
val = MyMassProp.Density
Debug.Print "Density: " & val
val = MyMassProp.SurfaceArea
Debug.Print "Surface area: " & val

pmoi = MyMassProp.PrincipalMomentsOfInertia
Debug.Print "Principal moments of inertia: Px: " & pmoi(0) & ", Py: " & pmoi(1) & ", and Pz: " & pmoi(2)

vValue = MyMassProp.GetMomentOfInertia(0)

Debug.Print "Moments of inertia: Lxx: " & vValue(0) & ", Lxy: " & vValue(1) & ", Lxz: " & vValue(2) & ", Lyx: " & vValue(3) & ", Lyy: " & vValue(4) & ", Lyz: " & vValue(5) & ", Lzx: " & vValue(6) & ", Lzy: " & vValue(7)
& ", Lzz: " & vValue(8)

End Sub
```

Get Mate Definition Example (VBA)

This example shows how to retrieve the mate definition.

```
' -----
' Preconditions:
' 1. Open an assembly document containing mates.
' 2. Select a mate.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the mate definition.
' 2. Examine the Immediate window.
' -----
Option Explicit

Sub ProcessDisplayDimension(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swDispDim As SldWorks.DisplayDimension, nVarFactor As Double, sVarType As String)
    Dim swDim As SldWorks.Dimension
    Dim vDimValueArr As Variant

    If Nothing Is swDispDim Then
        Debug.Print "    No display dimension"
        Debug.Print " "
        Exit Sub
    End If

    Set swDim = swDispDim.GetDimension

    vDimValueArr = swDim.GetSystemValue3(swThisConfiguration, Empty)
    Debug.Assert Not IsEmpty(vDimValueArr)
    Debug.Assert 0 = UBound(vDimValueArr)

    ' Show name and value of dimension
    Debug.Print "    Dim Name      = " & swDim.FullName
    Debug.Print "    Dim Value     = " & vDimValueArr(0) * nVarFactor & sVarType
    Debug.Print " "

End Sub

Sub ProcessMateEntity(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swMateEnt As SldWorks.MateEntity2)
    Dim swComp As SldWorks.Component2
    Dim vEntParam As Variant

    If Nothing Is swMateEnt Then
        Debug.Print "    No mate entity"
        Debug.Print " "
        Exit Sub
    End If

    Set swComp = swMateEnt.ReferenceComponent

    vEntParam = swMateEnt.EntityParams
    Debug.Assert Not IsEmpty(vEntParam)
    Debug.Assert 7 = UBound(vEntParam)

    ' Show name of mate component entity and its parameters
    Debug.Print "    Ref Comp = " & swComp.Name2
    Debug.Print "    Location = (" & vEntParam(0) * 1000# & ", " & vEntParam(1) * 1000# & ", " & vEntParam(2) * 1000# & ") mm"
    Debug.Print "    (i, j, k) = (" & vEntParam(3) & ", " & vEntParam(4) & ", " & vEntParam(5) & ") "
    Debug.Print "    Radius 1 = " & vEntParam(6) * 1000# & " mm"
    Debug.Print "    Radius 2 = " & vEntParam(7) * 1000# & " mm"
    Debug.Print " "

End Sub

Sub ProcessMate(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swMate As SldWorks.Mate2)
    Dim sVarType As String
    Dim nVarFactor As Double
    Dim nNumMateEnt As Long
    Dim i As Long

    ' Get the number of entities for the selected mate
    nNumMateEnt = swMate.GetMateEntityCount

    ' If mate type is Angle or Distance, then set conversion values
    Select Case swMate.Type
        Case swMate.ANGLE
            ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
            sVarType = " deg"
            nVarFactor = 57.3
        Case swMate.DISTANCE
            sVarType = " mm"
            nVarFactor = 1000#
    End Select

    ' Show if mate is aligned and can be flpped
    Debug.Print "    Alignment      = " & swMate.Alignment
    Debug.Print "    CanBeFlipped   = " & swMate.CanBeFlipped
End Sub
```

```

' If mate is Angle or Distance, show variances
If swMateANGLE = swMate.Type Or swMateDISTANCE = swMate.Type Then
    Debug.Print "    MaximumVariation    = " & swMate.MaximumVariation * nVarFactor & sVarType
    Debug.Print "    MinimumVariation    = " & swMate.MinimumVariation * nVarFactor & sVarType
End If

' Show type of mate
Debug.Print "    Type                = " & swMate.Type

ProcessDisplayDimension swApp, swModel, swMate.DisplayDimension, nVarFactor, sVarType

For i = 0 To nNumMateEnt - 1
    ProcessMateEntity swApp, swModel, swMate.MateEntity(i)
Next i

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swMate As SldWorks.Mate2
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swMate = swFeat.GetSpecificFeature2
    Debug.Print "File = " & swModel.GetPathName

    ' Show name of selected feature (in this case, a mate) and name of feature
    Debug.Print "    " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    ProcessMate swApp, swModel, swMate

End Sub

```

Get Mate Reference Properties Example (C#)

This example shows how to get mate reference properties.

```
//-----
// Preconditions:
// 1. Verify that the specified part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Inserts a mate reference.
// 3. Gets properties of mate reference.
// 4. Examine the FeatureManager design tree and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            MateReference swMateReference = default(MateReference);
            Feature swFeature = default(Feature);
            object mateRefObj = null;
            int mateRefEntityType = 0;
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Entity swPlane = default(Entity);
            FeatureManager swFeatureMgr = default(FeatureManager);
            string strMateReferencename = null;
            int nCount = 0;
            int refEntType = 0;
            int mateRefAlignment = 0;
            bool boolstatus = false;
            string fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\fillets\\knob.sldprt";
            int errors = 0;
            int warnings = 0;

            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Insert mate reference
            boolstatus = swModelDocExt.SelectByID2("Front", "PLANE", 0, 0, 0, true, 1, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swPlane = (Entity)swSelMgr.GetSelectedObject6(1, -1);
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.00835786916030656, 0.00429540237419701, 0, true, 2, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.InsertMateReference2("Default", null, 0, 1, false, null, 0, 2, false, null, 0, 0);
            swModel.ClearSelection2(true);
            boolstatus = swModelDocExt.SelectByID2("Default-1", "POSGROUP", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swMateReference = (MateReference)swFeature.GetSpecificFeature2();
            swModel.ClearSelection2(true);

            // Get the name of the mate reference
            strMateReferencename = swMateReference.Name;
            Debug.Print("Name of mate reference = " + strMateReferencename);

            // Get the number of reference entities in the mate reference
            nCount = swMateReference.ReferenceEntityCount;
            Debug.Print("Number of mate reference entities = " + nCount);

            // Get the mate reference type for the primary mate
            // entity in the selected mate reference
            refEntType = swMateReference.get_ReferenceType(0);
            Debug.Print("Mating type of primary mate entity = " + refEntType);

            // Get the mate reference alignment for the
            // mate reference entity in the selected mate reference
            mateRefAlignment = swMateReference.get_ReferenceAlignment(0);
            Debug.Print("Alignment of primary mate entity = " + mateRefAlignment);

            // Get the mate reference entity in the mate reference
            mateRefObj = swMateReference.get_ReferenceEntity2(0);

            // Get the mate reference entity type
            mateRefEntityType = swMateReference.get_ReferenceEntityType(0);
            Debug.Print("Entity type of primary mate entity = " + mateRefEntityType);

            // QueryInterface the returned object as a face, if a face
            if (mateRefEntityType == (int)swSelectType_e.swSelFACES)
            {
                Face2 mateRefFace = default(Face2);
                mateRefFace = (Face2)mateRefObj;

                Debug.Print("Primary mate entity is a face with area = " + mateRefFace.GetArea());
            }

            swModel.ClearSelection2(true);
        }
    }
}
```

```
    public SldWorks swApp;  
}  
}
```

Get Mate Reference Properties Example (VBA)

This example shows how to get mate reference properties.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Inserts a mate reference.
' 3. Gets properties of the mate reference.
' 4. Examine the FeatureManager design tree and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swMateReference As SldWorks.MateReference
    Dim swFeature As SldWorks.Feature
    Dim mateRefObj As Object
    Dim mateRefEntityType As Long
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swPlane As SldWorks.Entity
    Dim swFeatureMgr As SldWorks.FeatureManager
    Dim strMateReferencename As String
    Dim nCount As Long
    Dim refEntType As Long
    Dim mateRefAlignment As Long
    Dim boolstatus As Boolean
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    'Insert mate reference
    boolstatus = swModelDocExt.SelectByID2("Front", "PLANE", 0, 0, 0, True, 1, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swPlane = swSelMgr.GetSelectedObject6(1, -1)
    boolstatus = swModelDocExt.SelectByID2("", "FACE", 8.35786916030656E-03, 4.29540237419701E-03, 0, True, 2, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.InsertMateReference2("Default", Nothing, 0, 1, False, Nothing, 0, 2, False, Nothing, 0, 0)
    swModel.ClearSelection2 True

    boolstatus = swModelDocExt.SelectByID2("Default-1", "POSGROUP", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swMateReference = swFeature.GetSpecificFeature2
    swModel.ClearSelection2 True

    ' Get the name of the mate reference
    strMateReferencename = swMateReference.Name
    Debug.Print "Name of mate reference = " & strMateReferencename

    nCount = swMateReference.ReferenceEntityCount
    Debug.Print "Number of mate reference entities = " & nCount

    refEntType = swMateReference.ReferenceType(0)
    Debug.Print "Mating type of primary mate entity is " & refEntType

    mateRefAlignment = swMateReference.ReferenceAlignment(0)
    Debug.Print "Alignment of primary mate entity = " & mateRefAlignment

    ' Get the mate reference entity in the mate reference
    Set mateRefObj = swMateReference.ReferenceEntity2(0)

    ' Get the mate reference entity type
    mateRefEntityType = swMateReference.ReferenceEntityType(0)
    Debug.Print "Entity type of primary mate entity = " & mateRefEntityType

    ' QueryInterface the returned object as a face, if a face
    If mateRefEntityType = swSelectType_e.swSelFACES Then
        Dim mateRefFace As SldWorks.Face2
        Set mateRefFace = mateRefObj
        Debug.Print "Primary mate entity is a face with area = " & mateRefFace.GetArea
    End If

    swModel.ClearSelection2 True

End Sub

```

Get Mate Reference Properties Example (VB.NET)

This example shows how to get mate reference properties.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Inserts a mate reference.
' 3. Gets properties of the mate reference.
' 4. Examine the FeatureManager design tree and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swMateReference As MateReference
        Dim swFeature As Feature
        Dim mateRefObj As Object
        Dim mateRefEntityType As Long
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swPlane As Entity
        Dim swFeatureMgr As FeatureManager
        Dim strMateReferencename As String
        Dim nCount As Integer
        Dim refEntType As Integer
        Dim mateRefAlignment As Integer
        Dim boolstatus As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        swModel = swApp.ActiveDoc
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Insert mate reference
        boolstatus = swModelDocExt.SelectByID2("Front", "PLANE", 0, 0, 0, True, 1, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swPlane = swSelMgr.GetSelectedObject6(1, -1)
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.00835786916030656, 0.00429540237419701, 0, True, 2, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.InsertMateReference2("Default", Nothing, 0, 1, False, Nothing, 0, 2, False, Nothing, 0, 0)
        swModel.ClearSelection2(True)
        boolstatus = swModelDocExt.SelectByID2("Default<-1>", "POSGROUP", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swMateReference = swFeature.GetSpecificFeature2

        swModel.ClearSelection2(True)

        ' Get the name of the mate reference
        strMateReferencename = swMateReference.Name
        Debug.Print("Name of mate reference = " & strMateReferencename)

        ' Get the number of reference entities in the mate reference
        nCount = swMateReference.ReferenceEntityCount
        Debug.Print("Number of mate reference entities = " & nCount)

        ' Get the mate reference type for the primary mate
        ' entity in the selected mate reference
        refEntType = swMateReference.ReferenceType(0)
        Debug.Print("Mating type of primary mate entity = " & refEntType)

        ' Get the mate reference alignment for the
        ' mate reference entity in the selected mate reference
        mateRefAlignment = swMateReference.ReferenceAlignment(0)
        Debug.Print("Alignment of primary mate entity = " & mateRefAlignment)

        ' Get the mate reference entity in the mate reference
        mateRefObj = swMateReference.ReferenceEntity2(0)

        ' Get the mate reference entity type
        mateRefEntityType = swMateReference.ReferenceEntityType(0)
        Debug.Print("Entity type of primary mate entity = " & mateRefEntityType)

        ' QueryInterface the returned object as a face, if a face
        If mateRefEntityType = swSelectType_e.swSelFACES Then
            Dim mateRefFace As Face2
            mateRefFace = mateRefObj
            Debug.Print("Primary mate entity is a face with area = " & mateRefFace.GetArea)
        End If

        swModel.ClearSelection2(True)

    End Sub

    Public swApp As SldWorks

End Class

```

Get Material Database and XML Schema File Names Example (VBA)

This example shows how to get the names of the the path of the XML material schema file and the names of the material databases.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the path of the XML material schema file  
'    and the names of the material databases.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim vMatDBarr As Variant  
    Dim vMatDB As Variant  
  
    Set swApp = Application.SldWorks  
    vMatDBarr = swApp.GetMaterialDatabases  
    Debug.Print "Material schema path name = " & swApp.GetMaterialSchemaPathName  
    For Each vMatDB In vMatDBarr  
        Debug.Print " " & vMatDB  
    Next  
  
End Sub
```

Get Material Database Paths of Document Example (VBA)

This example shows how to get the paths and names of the material databases for a part.

NOTE: Material database names must be unique. Do not re-use the name of a material database.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\fillets\knob.sldprt.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the name of the part's material databases.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim dbs As Variant  
    Dim sMatName As String  
    Dim sMatDB As String  
    Dim bRet As Boolean  
    Dim i As Long  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
  
    dbs = swApp.GetMaterialDatabases  
  
    sMatName = swPart.GetMaterialPropertyName2("Default", sMatDB)  
    For i = 0 To LBound(dbs)  
        If StrComp(Left(Right(dbs(i), Len(sMatDB) + 7), Len(sMatDB)), sMatDB) = 0 Then  
            Debug.Print dbs(i)  
        End If  
    Next i  
  
End Sub
```

Get Material Example (VBA)

This example shows how to get the material for a part.

```
'-----  
' Preconditions:  
' 1. Open a part with a configuration named Default.  
' 2. Apply a material to the part.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the name of the material applied to the part.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim sMatName As String  
    Dim sMatDB As String  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
  
    sMatName = swPart.GetMaterialPropertyName2("Default", sMatDB)  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "   Material = " & sMatName & " (" & sMatDB & ")"  
  
End Sub
```

Get Material Properties Example (VBA)

This example shows how to iterate over the faces and features of a model and get the displayed colors.

NOTE: When in shaded mode, colors can be allocated, in order of priority, at the model, feature, and face level. To correctly get the color of a face, you must examine the color of the face at all of these levels. Also, several document-level preferences control how color is used. This example shows how to iterate over the faces and features of a model and correctly get the displayed color.

```

' -----
' Preconditions:
' 1. Open a part or fully resolved assembly.
' 2. If you opened an assembly, select any entity in the
'    the assembly.
' 3. Verify that c:\temp exists.
'
' Postconditions:
' 1. Writes the output to c:\ComponentMaterialPropertyValues.txt.
' 2. Examine c:\ComponentMaterialPropertyValues.txt.
' -----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swPart As SldWorks.PartDoc
    Dim swComp As SldWorks.Component2
    Dim swBody As SldWorks.Body2
    Dim swFace As SldWorks.Face2
    Dim swFeat As SldWorks.Feature
    Dim vMatProp As Variant
    Dim vConfigNames As Variant
    Dim i As Long
    Dim bRet As Boolean

    ' Open output file for component material property values
    Open "c:\temp\ComponentMaterialPropertyValues.txt" For Output As #1

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    If swModel.GetType = swDocPART Then
        Set swPart = swModel
        Set swBody = swPart.Body
    Else
        Set swComp = swSelMgr.GetSelectedObjectsComponent2(1)
        Set swBody = swComp.GetBody
    End If

    Set swFace = swBody.GetFirstFace

    ' Get document default colors and settings
    Write #1, "Model Default [" & swModel.GetPathName & "]"
    Write #1, " IgnoreFeatureColors = " & swModel.GetUserPreferenceToggle(swIgnoreFeatureColors)
    Write #1, " ColorRef = " & swModel.GetUserPreferenceIntegerValue(swDocumentColorShading)
    Write #1, ""

    vMatProp = swModel.MaterialPropertyValues
    Write #1, " RGB = [" & vMatProp(0) & ", " & vMatProp(1) & ", " & vMatProp(2) & "]"
    Write #1, " Ambient = " & vMatProp(3)
    Write #1, " Diffuse = " & vMatProp(4)
    Write #1, " Specular = " & vMatProp(5)
    Write #1, " Shininess = " & vMatProp(6)
    Write #1, " Transparency = " & vMatProp(7)
    Write #1, " Emission = " & vMatProp(8)
    Write #1, ""

    If Not swComp Is Nothing Then
        ' Component colors can override face colors
        vMatProp = swComp.MaterialPropertyValues
        If Not IsEmpty(vMatProp) Then
            Write #1, "Component Default [" & swComp.GetPathName & "]"
            Write #1, " RGB = [" & vMatProp(0) & ", " & vMatProp(1) & ", " & vMatProp(2) & "]"
            Write #1, " Ambient = " & vMatProp(3)
            Write #1, " Diffuse = " & vMatProp(4)
            Write #1, " Specular = " & vMatProp(5)
            Write #1, " Shininess = " & vMatProp(6)
            Write #1, " Transparency = " & vMatProp(7)
            Write #1, " Emission = " & vMatProp(8)
            Write #1, ""
        End If
    End If

    While Not swFace Is Nothing
        ' Face colors can override part colors
        vMatProp = swFace.MaterialPropertyValues
        If Not IsEmpty(vMatProp) Then
            Write #1, "Face[" & i & "] overridden"
            Write #1, " RGB = [" & vMatProp(0) & ", " & vMatProp(1) & ", " & vMatProp(2) & "]"
            Write #1, " Ambient = " & vMatProp(3)
        End If
        i = i + 1
        Set swFace = swBody.GetNextFace(swFace)
    End While
End Sub

```

```

        Write #1, " Diffuse      = " & vMatProp(4)
        Write #1, " Specular    = " & vMatProp(5)
        Write #1, " Shininess   = " & vMatProp(6)
        Write #1, " Transparency = " & vMatProp(7)
        Write #1, " Emission    = " & vMatProp(8)
        Write #1, ""
    End If

    ' Get feature associated with this face
    Set swFeat = swFace.GetFeature
    vConfigNames = swModel.GetConfigurationNames

    ' Face colors can override feature colors
    vMatProp = swFeat.GetMaterialPropertyValues2(swSpecifyConfiguration, (vConfigNames))

    If Not IsEmpty(vMatProp) Then
        Write #1, "Face[" & i & "] overridden by "; swFeat.Name
        Write #1, " RGB          = [" & vMatProp(0) & ", " & vMatProp(1) & ", " & vMatProp(2) & "]"
        Write #1, " Ambient       = " & vMatProp(3)
        Write #1, " Diffuse      = " & vMatProp(4)
        Write #1, " Specular    = " & vMatProp(5)
        Write #1, " Shininess   = " & vMatProp(6)
        Write #1, " Transparency = " & vMatProp(7)
        Write #1, " Emission    = " & vMatProp(8)
        Write #1, ""
    End If
    i = i + 1
    Set swFace = swFace.GetNextFace
Wend

    ' Close output file for component material property values
    Close

End Sub

```

Get Material Property Names Example (VBA)

This example shows how to get the names of the materials applied to a part and faces of the part.

```
'-----  
' Preconditions:  
' 1. Open a part to which materials are applied  
'    to multiple faces.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the name of the material applied to  
'    the part and to each face of the part.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc  
    Dim swPart As SldWorks.PartDoc  
    Dim swConfig As SldWorks.Configuration  
    Dim swBody As SldWorks.Body  
    Dim swFace As SldWorks.Face  
    Dim sMatName As String  
    Dim sMatUserName As String  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swConfig = swModel.GetActiveConfiguration  
  
    Set swBody = swPart.Body  
    Set swFace = swBody.GetFirstFace  
    Debug.Print "SOLIDWORKS revision number  = " & swApp.RevisionNumber  
    Debug.Print "  File = " & swModel.GetPathName  
    Debug.Print "    Configuration name      = " & swConfig.Name  
    Debug.Print "    Part material name          = " & swPart.MaterialIdName  
    Debug.Print "    Part material user name       = " & swPart.MaterialUserName  
    Debug.Print ""  
  
    Do While Not swFace Is Nothing  
        sMatName = swFace.MaterialIdName  
        sMatUserName = swFace.MaterialUserName  
        If sMatName <> "" Then  
            Debug.Print "    Face material name (" & i & ")      = " & sMatName  
        End If  
        If sMatUserName <> "" Then  
            Debug.Print "    Face material user name (" & i & ") = " & sMatUserName  
        End If  
        i = i + 1  
        Set swFace = swFace.GetNextFace  
    Loop  
  
End Sub
```

Get Material Property Values Example (VBA)

This example shows how to get the color, a material property value, applied to each body in a part.

```
'-----
' Preconditions:
' 1. Open a part to which color is applied
'    to one or more bodies.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the color on each body in a part.
' 2. Examine the Immediate window.
'-----
Option Explicit

Public Enum swBodyType_e
    swSolidBody = 0
    swSheetBody = 1
    swWireBody = 2
    swMinimumBody = 3
    swGeneralBody = 4
    swEmptyBody = 5
End Enum

Sub ProcessBodyArray(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vBodyArr As Variant)
    Dim vBody As Variant
    Dim swBody As SldWorks.Body2
    Dim vMatProp As Variant

    For Each vBody In vBodyArr
        Set swBody = vBody
        vMatProp = swBody.MaterialPropertyValues2
        If IsEmpty(vMatProp) Then
            Debug.Print "    No color information."
        Else
            Debug.Print "    RGB          = [" & vMatProp(0) * 255# & ", " & vMatProp(1) * 255# & ", " & vMatProp(2) * 255# & "]"
            Debug.Print "    Ambient      = " & vMatProp(3)
            Debug.Print "    Diffuse      = " & vMatProp(4)
            Debug.Print "    Specular     = " & vMatProp(5)
            Debug.Print "    Shininess    = " & vMatProp(6)
            Debug.Print "    Transparency = " & vMatProp(7)
            Debug.Print "    Emission     = " & vMatProp(8)
            Debug.Print ""
        End If
    Next
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim vBodyArr As Variant
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel

    Debug.Print "File = " & swModel.GetPathName

    For i = 0 To 5
        vBodyArr = swPart.GetBodies2(i, False)
        If Not IsEmpty(vBodyArr) Then
            Debug.Print "    Body[" & i & "] = " & UBound(vBodyArr) + 1
            ProcessBodyArray swApp, swModel, vBodyArr
        End If
    Next i
End Sub
```

Get Mates and Mate Entities Example (C#)

This example shows how to iterate through a FeatureManager design tree and get all of an assembly's mates and mate entities.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Iterates through the FeatureManager design tree to find
//    and print Mates to the Immediate window.
// 3. Iterates through the subfeatures of Mates and prints each mate
//    entity's information to the Immediate window.
// 4. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;

namespace Mate2CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeat = default(Feature);
            Feature swMateFeat = null;
            Feature swSubFeat = default(Feature);
            Mate2 swMate = default(Mate2);
            Component2 swComp = default(Component2);
            MateEntity2[] swMateEnt = new MateEntity2[3];
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            int i = 0;
            double[] entityParameters = new double[8];

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\simple-block.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, ""
, ref errors, ref warnings);

            //Get the first feature in the assembly
            swFeat = (Feature)swModel.FirstFeature();
            //Iterate over features in FeatureManager design tree
            while ((swFeat != null))
            {
                if ("MateGroup" == swFeat.GetTypeName())
                {
                    {
                        swMateFeat = (Feature)swFeat;
                        break;
                    }
                    swFeat = (Feature)swFeat.GetNextFeature();
                }
                Debug.Print(" " + swMateFeat.Name);
                Debug.Print("");

                //Get first mate, which is a subfeature
                swSubFeat = (Feature)swMateFeat.GetFirstSubFeature();
                while ((swSubFeat != null))
                {
                    swMate = (Mate2)swSubFeat.GetSpecificFeature2();
                    if ((swMate != null))
                    {
                        for (i = 0; i <= 1; i++)
                        {
                            {
                                swMateEnt[i] = swMate.MateEntity(i);
                                Debug.Print(" " + swSubFeat.Name);
                                Debug.Print("      Type          = " + swMate.Type);
                                Debug.Print("      Alignment     = " + swMate.Alignment);
                                Debug.Print("      Can be flipped = " + swMate.CanBeFlipped);
                                Debug.Print("");
                                swComp = (Component2)swMateEnt[i].ReferenceComponent;
                                Debug.Print("      Component     = " + swComp.Name2);
                                Debug.Print("      Mate entity type = " + swMateEnt[i].ReferenceType);
                                entityParameters = (double[])swMateEnt[i].EntityParams;
                                Debug.Print("      (x,y,z)");
                                = (" + (double)entityParameters[0] + ", " + (double)entityParameters[1] + ", " + (double)entityParameters[2] + ")");
                                Debug.Print("      (i,j,k)");
                                = (" + (double)entityParameters[3] + ", " + (double)entityParameters[4] + ", " + (double)entityParameters[5] + ")");
                                Debug.Print("      Radius 1      = " + (double)entityParameters[6]);
                                Debug.Print("      Radius 2      = " + (double)entityParameters[7]);
                                Debug.Print("");
                            }
                            Debug.Print(" ");
                        }
                        // Get the next mate in MateGroup
                        swSubFeat = (Feature)swSubFeat.GetNextSubFeature();
                    }
                }
            }

            /// <summary>
            /// The SldWorks swApp variable is pre-assigned for you.
            /// </summary>
            public SldWorks swApp;
        }
    }
}
```

Get Mates and Mate Entities Example (VBA)

This example shows how to iterate through a FeatureManager design tree and get all of an assembly's mates and mate entities.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Iterates through the FeatureManager design tree find
'    and print Mates feature to the Immediate window.
' 3. Iterates through the subfeatures of Mates and prints each mate
'    entity's information to the Immediate window.
' 4. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeat As SldWorks.Feature
Dim swMateFeat As SldWorks.Feature
Dim swSubFeat As SldWorks.Feature
Dim swMate As SldWorks.Mate2
Dim swComp As SldWorks.Component2
Dim swMateEnt(2) As SldWorks.MateEntity2
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\simple-block.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Get the first feature in the assembly
    Set swFeat = swModel.FirstFeature

    'Iterate over features in FeatureManager design tree
    Do While Not swFeat Is Nothing
        If "MateGroup" = swFeat.GetTypeName Then
            Set swMateFeat = swFeat
            Exit Do
        End If
        Set swFeat = swFeat.GetNextFeature
    Loop

    Debug.Print " " & swMateFeat.Name
    Debug.Print ""

    'Get first mate, which is a subfeature
    Set swSubFeat = swMateFeat.GetFirstSubFeature
    Do While Not swSubFeat Is Nothing
        Set swMate = swSubFeat.GetSpecificFeature2
        If Not swMate Is Nothing Then
            For i = 0 To 1
                Set swMateEnt(i) = swMate.MateEntity(i)
                Debug.Print " " & swSubFeat.Name
                Debug.Print "    Type          = " & swMate.Type
                Debug.Print "    Alignment     = " & swMate.Alignment
                Debug.Print "    Can be flipped = " & swMate.CanBeFlipped
                Debug.Print ""
                Set swComp = swMateEnt(i).ReferenceComponent
                Debug.Print "    Component     = " & swComp.Name2
                Debug.Print "    Mate entity type = " & swMateEnt(i).ReferenceType
                Debug.Print "    (x,y,z)       = (" & swMateEnt(i).EntityParams(0) & ", " & swMateEnt(i).EntityParams(1) & ", " &
                swMateEnt(i).EntityParams(2) & ") "
                Debug.Print "    (i,j,k)       = (" & swMateEnt(i).EntityParams(3) & ", " & swMateEnt(i).EntityParams(4) & ", " &
                swMateEnt(i).EntityParams(5) & ") "
                Debug.Print "    Radius 1      = " & swMateEnt(i).EntityParams(6)
                Debug.Print "    Radius 2      = " & swMateEnt(i).EntityParams(7)
                Debug.Print ""
            Next i
            Debug.Print " "
        End If
        ' Get the next mate in MateGroup
        Set swSubFeat = swSubFeat.GetNextSubFeature
    Loop

End Sub

```

Get Mates and Mate Entities Example (VB.NET)

This example shows how to iterate through a FeatureManager design tree and get all of an assembly's mates and mate entities.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Iterates through the FeatureManager design tree to find
'    print Mates to the Immediate window.
' 3. Iterates through the subfeatures of Mates and prints each mate
'    entity's information to the Immediate window.
' 4. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swFeat As Feature
        Dim swMateFeat As Feature = Nothing
        Dim swSubFeat As Feature
        Dim swMate As Mate2
        Dim swComp As Component2
        Dim swMateEnt(2) As MateEntity2
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim i As Integer
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\simple-block.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Get the first feature in the assembly
        swFeat = swModel.FirstFeature
        'Iterate over features in FeatureManager design tree
        Do While Not swFeat Is Nothing
            If "MateGroup" = swFeat.GetTypeName Then
                swMateFeat = swFeat
                Exit Do
            End If
            swFeat = swFeat.GetNextFeature
        Loop
        Debug.Print(" " & swMateFeat.Name)
        Debug.Print("")

        'Get first mate, which is a subfeature
        swSubFeat = swMateFeat.GetFirstSubFeature
        Do While Not swSubFeat Is Nothing
            swMate = swSubFeat.GetSpecificFeature2
            If Not swMate Is Nothing Then
                For i = 0 To 1
                    swMateEnt(i) = swMate.MateEntity(i)
                    Debug.Print(" " & swSubFeat.Name)
                    Debug.Print("    Type           = " & swMate.Type)
                    Debug.Print("    Alignment      = " & swMate.Alignment)
                    Debug.Print("    Can be flipped = " & swMate.CanBeFlipped)
                    Debug.Print("")
                    swComp = swMateEnt(i).ReferenceComponent
                    Debug.Print("    Component      = " & swComp.Name2)
                    Debug.Print("    Mate entity type = " & swMateEnt(i).ReferenceType)
                    Debug.Print("    (x,y,z)        = " & swMateEnt(i).EntityParams(0) & ", " & swMateEnt(i).EntityParams(1) & ", " & swMateEnt(i).EntityParams(2) & ")")
                    Debug.Print("    (i,j,k)        = " & swMateEnt(i).EntityParams(3) & ", " & swMateEnt(i).EntityParams(4) & ", " & swMateEnt(i).EntityParams(5) & ")")
                    Debug.Print("    Radius 1       = " & swMateEnt(i).EntityParams(6))
                    Debug.Print("    Radius 2       = " & swMateEnt(i).EntityParams(7))
                    Debug.Print("")
                Next i
                Debug.Print(" ")
            End If
            ' Get the next mate in MateGroup
            swSubFeat = swSubFeat.GetNextSubFeature
        Loop
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```